

NPLify · P0 Technical Baseline

Deliverable 1 — Entity-Relationship Diagram & Data Model · Draft v5.3

New XP Technologies Limited

9 October 2026 · Confidential · Internal baseline document

Status and conventions

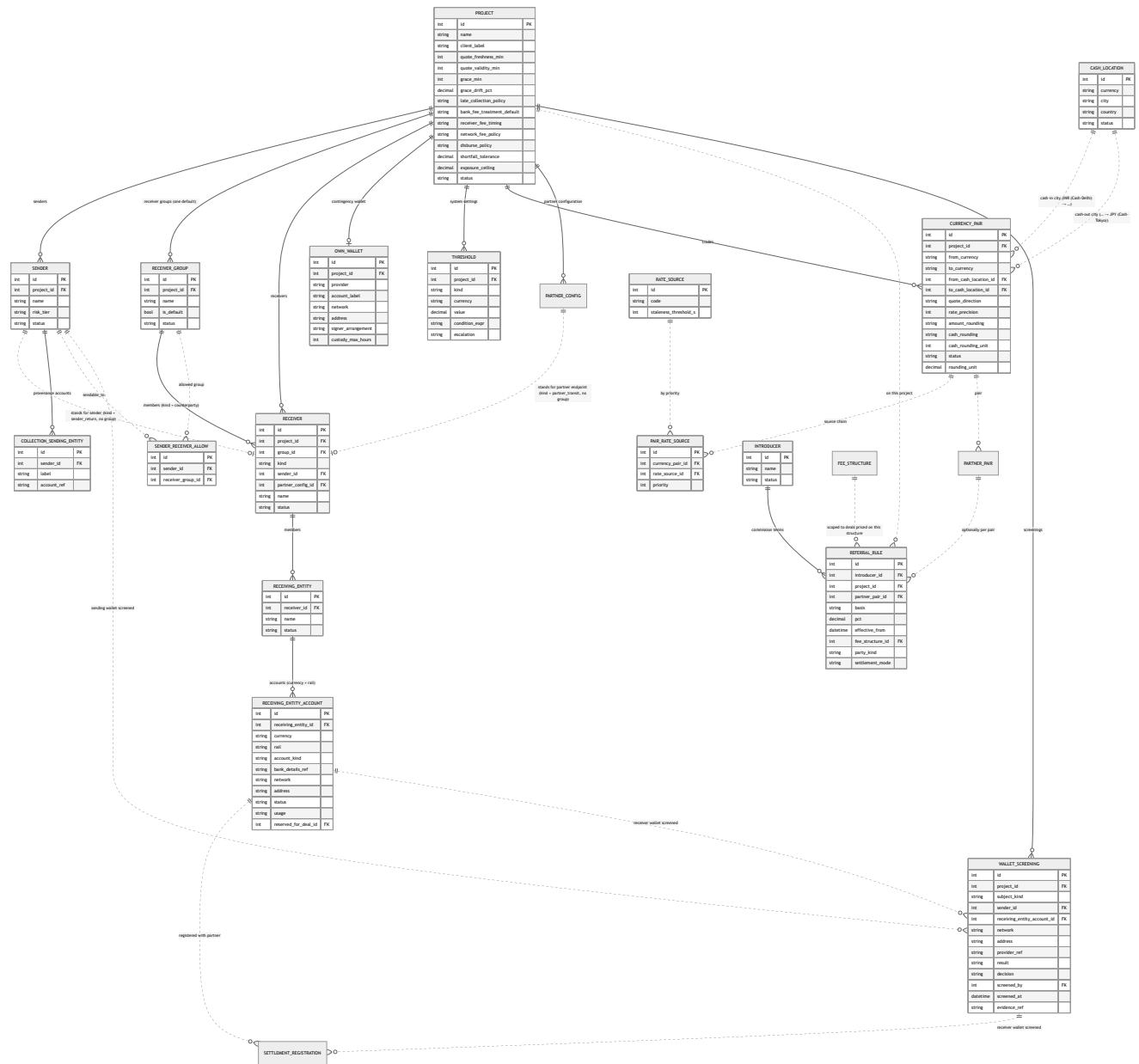
Status: Draft v5.3 (9 October 2026) applies the thirteen fee-practice decisions D20–D32 and the same-currency rule D33, confirmed by NPL on 8–9 October from the fee table, Sud's workbook answers and the Project Context and Payment methods tabs: volume tiers and per-sender splits (D20, D21), the displayed fee (D22), NPL-GR as an own-desk project (D23), invoices paid in parts (D24), a rounding menu (D25), markup shared with a client party and rebates as their own stream (D26, D27), partner rates as quoted (D28), the bank cutoff and rate lock (D29), cash by token with rates per city (D30), wallet screening (D31), single-use accounts and loss events (D32), and same-currency pass-through (D33). Six tables and two views are added; invariant 3 is restated; invariants 20–32 are new. The amendments of 9 October (D27, D29, D30) are recorded in §6. It carries v5.0–v5.2 (D13–D19) unchanged. Supersedes v5.2, v5.1, v5.0 and v4.0 in full. This baseline is an internal New XP document; the client deliverables are derived from it. Dependent documents have **not** yet been regenerated against this version (see §6).

Convention: all monetary amounts are stored as exact decimals; every table carries `created_at` / `created_by`; audit, approvals and locking are modelled once (domain 4) and apply to all master data. Derived figures (client balance, group entitlement, offsets, custody) are **views over postings and deals** — **never stored**.

Seven diagrams across four domains, one schema: 60 tables and 6 views. In each diagram, entities defined in another diagram appear with no attributes and name that diagram. Relationship lines run from the referenced row to the row that stores the foreign key; `||--o{` is composition (the parent owns the child), `||..o{` a reference or a polymorphic link resolved by a type column plus an id. Diagram 3a of v5.2 is split into 3a and 3c in v5.3 because the deal leg and its stamps outgrew one page. Section 6 lists what changed since v4.0; Appendix A reconciles the model with the FRS; Appendix B is the data dictionary.

1 · Configuration domain

A **Project** (= client) owns its configuration. Partners are global counterparties; how a partner is used *in this project* is the **partner configuration** block — fees, vehicles, endpoints, registrations and rebate terms hang off it. Counterparty receivers are organised in **receiver groups**, the unit entitlement is tracked on, and each receiving entity holds one or more **accounts** with a currency and rail.





Notes

- A **Project** (= client) owns its configuration. Counterparty receivers sit in **receiver groups**: every project has one default group and every counterparty receiver belongs to exactly one group. Entitlement is tracked per group, so payouts within a group are ordinary settlement and only payouts across groups are offsets; a project with a single group never has an offset.
- A **Receiver** has a kind: counterparty for a client receiver; sender_return stands for a sender so refunds use the ordinary payout path and the partner's destination approval; partner_transit stands for a partner's collection endpoint so the hop of a two-leg route is an ordinary payout. sender_return and partner_transit receivers belong to no group (group_id is null). They never appear on forward deals or allow lists — both validate kind = counterparty — and a payout to them is attributed to the group of the leg that pays them, so no entitlement moves between groups on a refund or a hop. Groups exist for entitlement and offset accounting; refund exclusion is a receiver-kind rule, not a group (D13).
- A receiving entity holds one or more **accounts**, each with a currency and rail (EUR via SWIFT and EUR via SEPA are two accounts). An account is a bank account or a wallet (account_kind): a bank account carries bank_details_ref and a bank rail; a wallet carries network and address explicitly, mirroring the partner-side COLLECTION_RECEIVING_ENDPOINT, so addresses can be checksum-validated, checked against the rail, and found when the same address is registered with two partners. Registrations and payout lines point at the account. No table stores actual bank details: bank_details_ref points at the external store.
- Fees are per partner × pair (FEE_STRUCTURE under PARTNER_PAIR), so choosing the partner is part of pricing. PARTNER.disclosure_style records whether the partner's markup is visible; partner_markup_pct is known only for market_plus partners.
- A partner operates through one or more legal vehicles (PARTNER_ENTITY: PT-tour, PT Global Inc); the deal stamps the vehicle at quote so paperwork names the right company. Each vehicle owns the endpoints a sender can pay into (COLLECTION_RECEIVING_ENDPOINT): kind = bank with a currency and bank_details_ref, or kind = wallet with network and address. The partner side therefore mirrors the client side — PARTNER_CONFIG → PARTNER_ENTITY → COLLECTION_RECEIVING_ENDPOINT as RECEIVER → RECEIVING_ENTITY → RECEIVING_ENTITY_ACCOUNT (D15). An endpoint also carries a nullable rail: a bank endpoint must have it before it becomes active, and a wallet may leave it null but must match network when it is set (D18); the endpoint's kind must match the leg's collection_method (D19). Cash-in has no endpoint record; the leg carries the city and the collection the token (D30). The collected cash amount is rounded by the pair's cash_rounding option — truncate, nearest or round up — to cash_rounding_unit (100 / 500 / 1,000 by currency), the same menu as payout rounding (D25).
- Destination approval is per partner: SETTLEMENT_REGISTRATION is an account of the project's receiving entity as registered with this partner, carrying the partner's approval status and the payout details the partner holds. Those details follow the account's kind: bank_details_ref for a bank account, network and address for a wallet (D16). SETTLEMENT_SENDING_ENTITY is the narrative on an outgoing transfer; SETTLEMENT_RAIL is the bank/EMI that executes it; neither holds money. COLLECTION_SENDING_ENTITY is provenance only and never a payout destination.
- OWN_WALLET exists for rerouting and pass-through (AQN-LT-Sub is an own wallet at Aquanow); PARTNER_CONFIG.applicable_for_reroute defines the alternate-partner set. Under an **own-desk** partner configuration (is_own_desk, the NPL-GR game-reseller project) NPL itself is the converting party: its pairs are priced market plus NPL's markup and its collection endpoints are NPL's own wallets (COLLECTION_RECEIVING_ENDPOINT.own_wallet_id); the custody view reports every such balance (D23).

- **Fees in v5.3.** A `FEE_OVERRIDE` may carry its own sender / receiver split instead of inheriting the structure's proportion (D20). `FEE_TIER` rows under a structure price by the project's calendar-month volume; a leg that takes the month past a threshold is split across two tiers (D21). `displayed_fee_basis` / `displayed_fee_pct` on the structure say what the customer's quote prints when it differs from the recorded parts; the printed "source rate" is derived at quote, never typed (D22). `CURRENCY_PAIR.amount_rounding` is a menu — truncate, nearest or round up, to the unit or to `rounding_unit` — and a pair may have the same currency on both sides (D25, D33).
- **Who else earns on a deal.** `INTRODUCER` / `REFERRAL_RULE` record a share NPL pays — or, with basis `share_of_net_markup` and `party_kind` `client_party`, the customer who provides a sub-account and shares NPL's markup (Raeen): the base is the leg's fee parts less its partner cost, rebates excluded, several parties may hold rules with different shares, and `settlement_mode` `party_retains` means the party holds the converted money and pays NPL the remainder monthly (D26). `PARTNER_REBATE_RULE` records the rebate a partner pays NPL, with a basis (percentage of amount or of the partner's fee) and an optional scope by sender or fee structure; the rebate is NPL's alone and is reported as its own stream (D27).
- **Cutoff, screening, single use.** `PARTNER.bank_cutoff_time` (the partner's own clock, beside its rate expiry and holiday calendar) and `PARTNER_PAIR.needs_bank_lock` drive same-day or next-day conversion (D29). A cash side of a currency pair names its `CASH_LOCATION` — INR (Cash-Delhi) → USDT and INR (Cash-Mumbai) → USDT are two pairs — so partner pairs, fee structures and rate days are per city by construction; bank and crypto sides carry no location (D30). `WALLET_SCREENING` records the screening of a sending or receiving wallet address, by Operations, and the endpoint's `risk_band` says which decisions may land on it (D31). An endpoint or a receiving-entity account may be `single_use`, reserved for one deal (ad hoc local deposit accounts, D32).

2 · Rates domain

Diagram 2. RATE_SOURCE, CURRENCY_PAIR are defined in Diagram 1a; PARTNER_PAIR in 1b; DEAL in 3a; CONVERSION, BALANCE_CONVERSION in 3c.

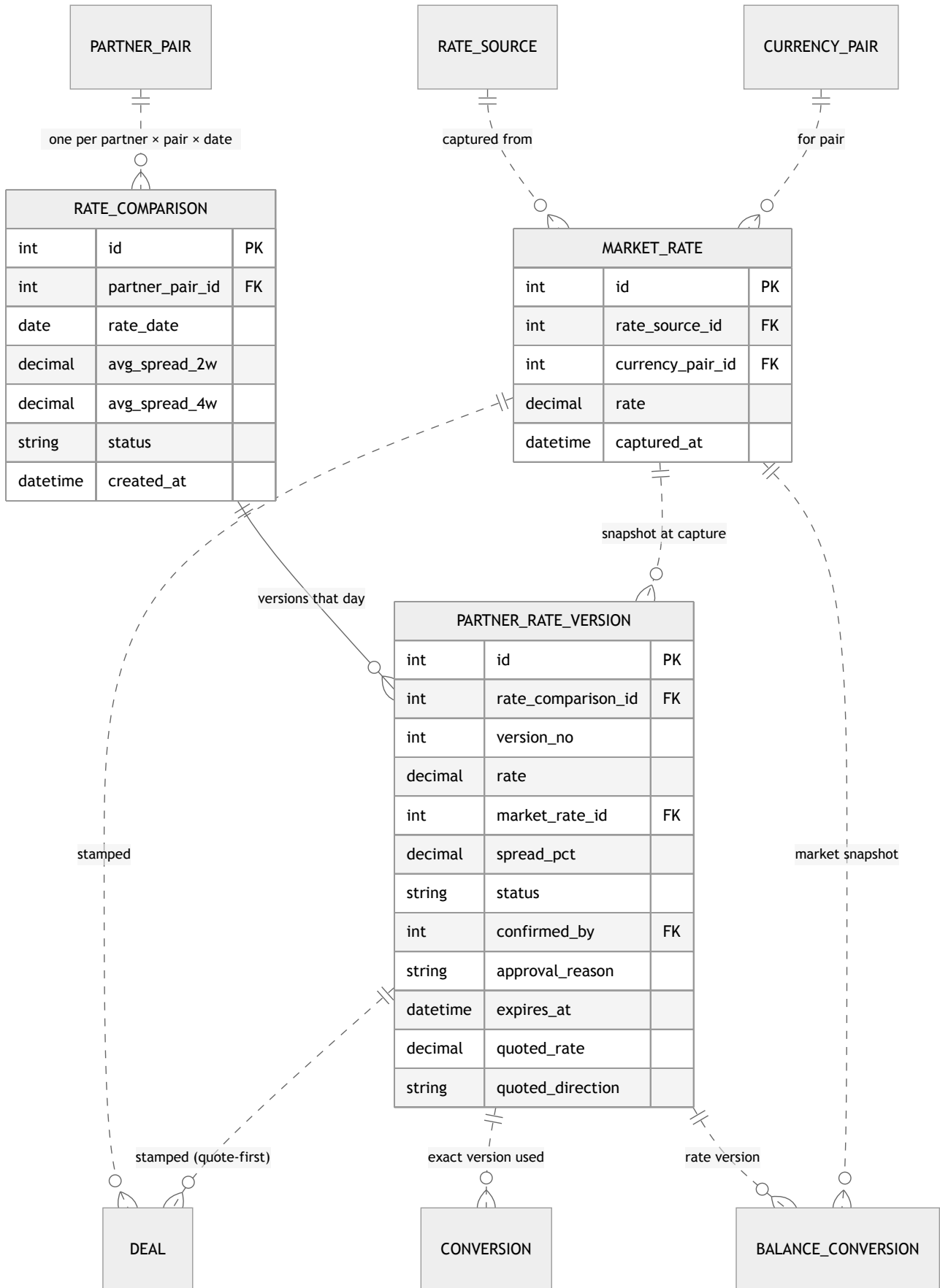


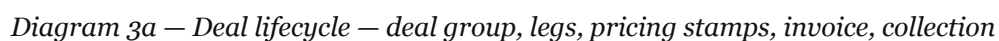
Diagram 2 — Rates

Notes

- The rate comparison for the day is created lazily by the first deal that needs it, one per partner × pair × date, and reused by every deal that day. A partner may give a new rate during the day, so one date can hold several versions. Rolling averages are built from confirmed versions only.
- A version is breached when $\text{spread} > \text{avg_2w} \times (1 + \text{variance_threshold_rel})$; quoting and conversion on that partner × pair freeze until Finance approves with a reason, rejects, or asks for a better rate. Versions are never overwritten; every deal leg, conversion and balance conversion references its exact version.
- Cross-source slippage is a query over MARKET_RATE across sources for the same pair and window.
- A version stores the rate **as the partner quoted it** (quoted_rate, quoted_direction); rate is normalised to the pair's direction and derived — $1 \div \text{quoted}$ when the directions differ — so Ali's 1.1738 becomes 0.851934 without a hand inversion, and a direction slip is caught by the breach check (D28). A cash city is its own currency pair (CASH_LOCATION), so Delhi and Mumbai have their own rate days and rolling averages (D30).

3 · Deal lifecycle domain

3a · Deal group, legs, pricing stamps, invoice, collection



3b · Disbursement, return, confirmation, fees, referrals, rebates and loss events

Diagram 3b. PROJECT, RECEIVER, REFERRAL_RULE are defined in Diagram 1a; PARTNER_CONFIG, SETTLEMENT_SENDING_ENTITY, SETTLEMENT_RAIL, SETTLEMENT_REGISTRATION, PARTNER_REBATE_RULE in 1b; MARKET_RATE in 2; DEAL, COLLECTION, INVOICE in 3a; BALANCE_CONVERSION in 3c.

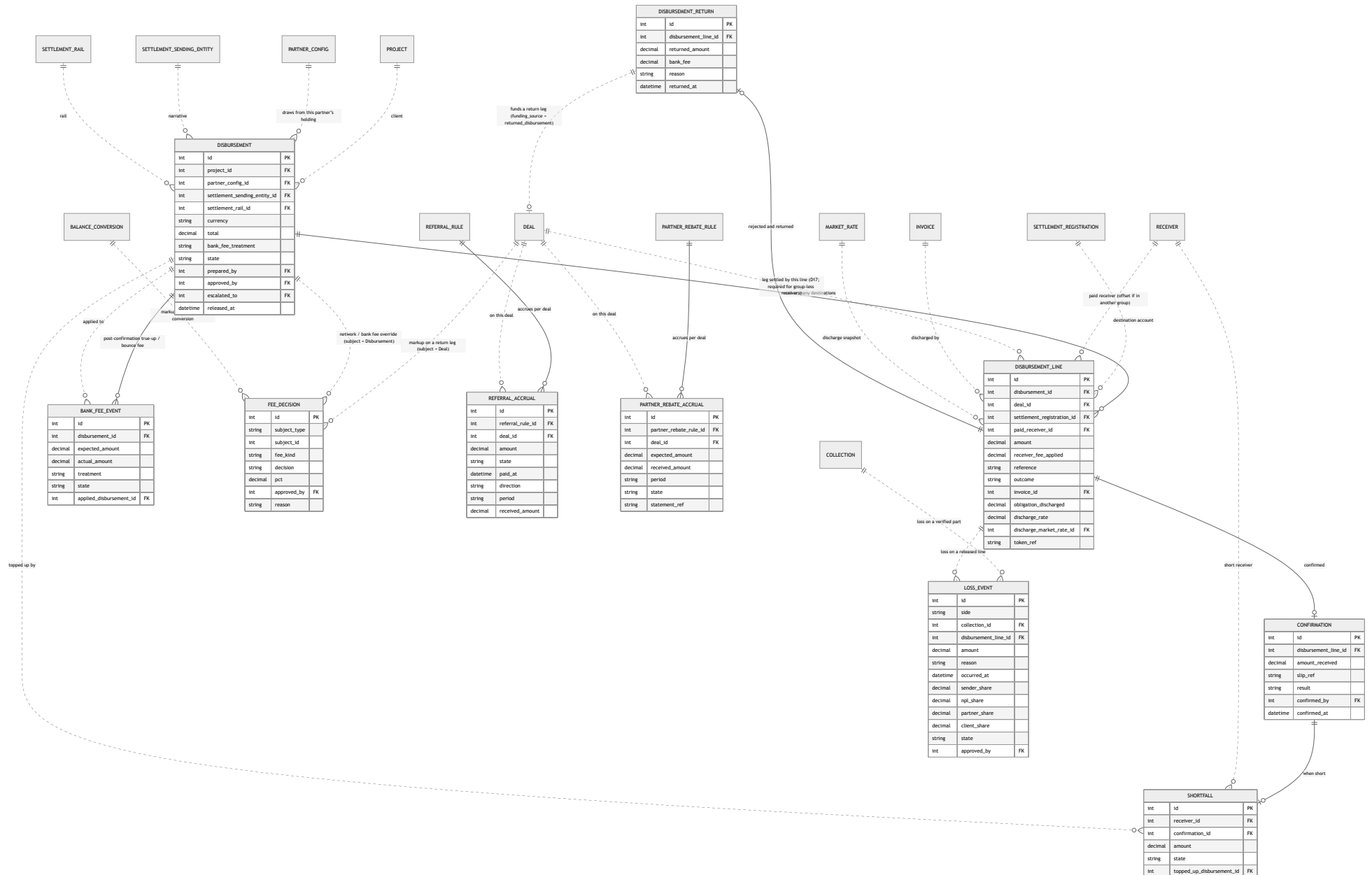


Diagram 3b — Disbursement, return, confirmation, fees, referrals, rebates and loss events

3c · Conversion, reroute, earnings receivable, balance conversion, entitlement re-attribution

Diagram 3c. PROJECT, RECEIVER_GROUP, OWN_WALLET are defined in Diagram 1a; PARTNER_CONFIG, PARTNER_PAIR in 1b; PARTNER_RATE_VERSION, MARKET_RATE in 2; DEAL in 3a.

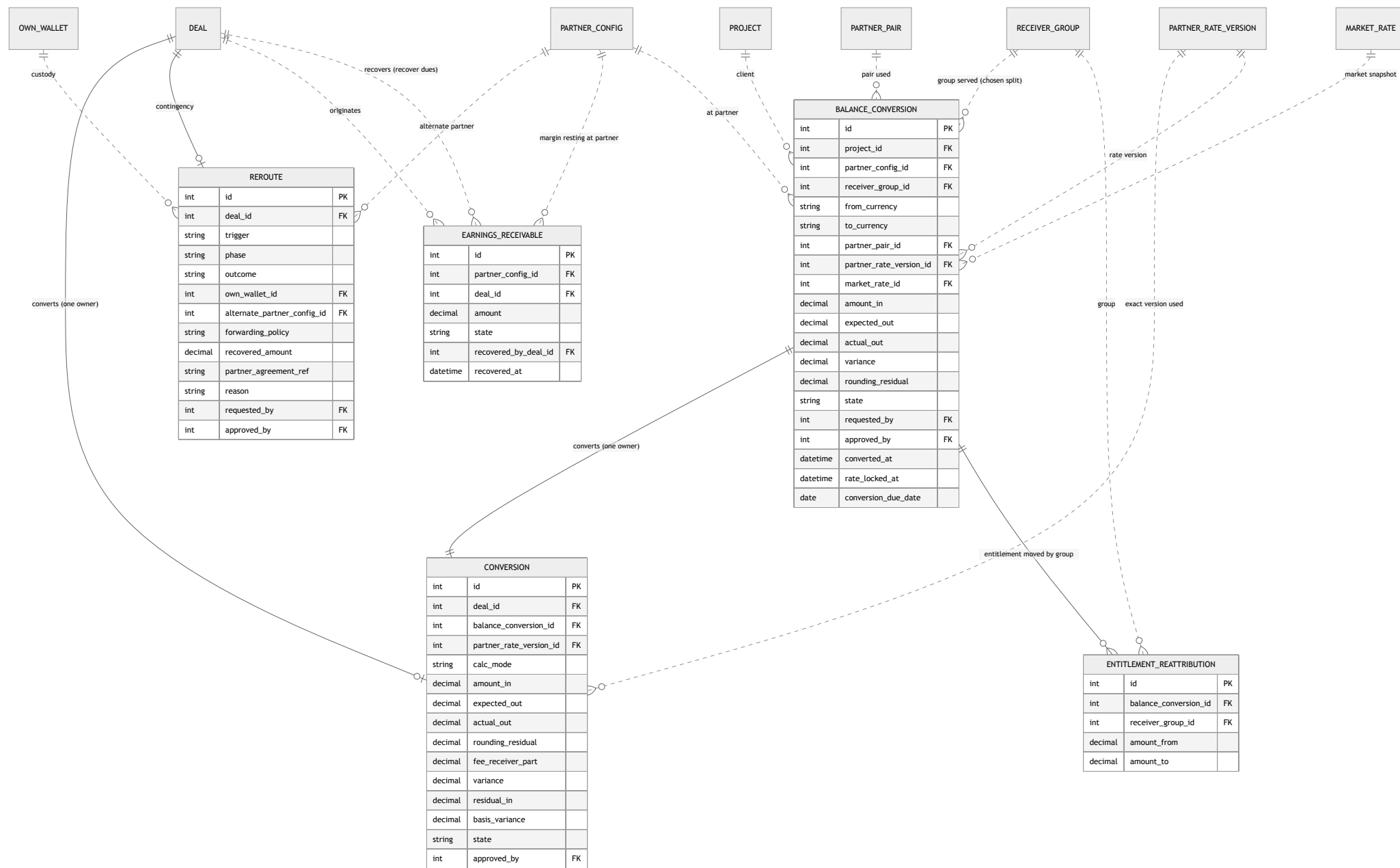


Diagram 3c — Deal lifecycle — conversion, reroute, earnings receivable, balance conversion, entitlement re-attribution

Derived views (never stored)

View	Definition
CLIENT_BALANCE(project, partner, currency)	CLIENT_BALANCE(project, partner, currency) = Σ conversions' actual_out booked at that partner (deal legs and balance conversions, target side) – Σ balance conversions' amount_in (source side) + Σ returned amounts – Σ released disbursement totals. Carries rounding dust below one unit. v5.3 (D29): also reports a pending column — the expected_out of legs whose rate is locked with the partner but whose money has not yet been collected or converted; a void removes it. Pending is never a posting.
GROUP_ENTITLEMENT(group, currency)	GROUP_ENTITLEMENT(group, currency) = Σ client_net of converted legs (forward and return) attributed to the group $\pm$ entitlement re-attributions – Σ DISBURSEMENT_LINE.amount on lines whose disbursing deal is attributed to the group + Σ returned amounts on those lines. A payout to a group-less receiver debits the group of the leg that pays it, read from DISBURSEMENT_LINE.deal_id (v5.0 D13; D17 makes the paying leg a stored column). Σ over a project's groups = Σ CLIENT_BALANCE over partners, per currency.
OFFSET(group, currency)	OFFSET(group, currency) = GROUP_ENTITLEMENT; positive = owed, negative = holding another group's flow. Arises only when a counterparty receiver is paid from a leg attributed to another group (v5.0); zero by construction in a single-group project. Carries Management approval and tighter aging.
CUSTODY(own_wallet)	CUSTODY(own_wallet) = Σ collections into the own wallet under open reroutes + Σ recovered dues resting there + Σ own-desk client balances (v5.3, D23) – Σ forwarded or paid out; each row with deal, reason, age. The standing control on money in NPL's hands.
INVOICE_BALANCE(invoice)	INVOICE_BALANCE(invoice) = INVOICE.amount – Σ DISBURSEMENT_LINE.obligation_discharged over lines with outcome confirmed. The open part of a vendor obligation paid in another currency, in parts (v5.3, D24).
PROJECT_MONTH_VOLUME(project, month, currency)	PROJECT_MONTH_VOLUME(project, month, currency) = Σ DEAL.amount_in over forward legs of the project collected in the calendar month, in the tier's volume currency. Read at quote to choose the FEE_TIER; stamped on the leg as tier_volume_basis (v5.3, D21).

Notes

- **Every deal belongs to a deal group.** DEAL_GROUP is the client-facing instruction: the pair as the sender sees it, the sender rate, fee, reference and entry mode. A plain single-partner deal is a group of one leg; a route that needs two partners (INR → USDT at Ali, USDT → EUR at Jeton) has two forward legs; a rejected payout is unwound with a return leg.
- **Legs.** leg_type forward or return; funding_source collection, prior_leg (leg 2 of a route, no collection) or returned_disbursement (a return leg). Priced once and partner fixed at quote hold per leg. Attribution is to the receiver group: set at quote on the first forward leg, inherited by leg n of a route from leg n–1 and by a return leg from its parent leg. The receiver is optional until settlement.
- Two entry modes as before: collect-first legs are born in state Collected with priced_at = conversion time; quote-first legs carry sent / valid / grace timestamps and a late decision. An estimate-first mode gives an indicative figure and prices at conversion.

- **Conversion has exactly one owner:** a deal leg or a `BALANCE_CONVERSION`. The receiver-side fee is taken at conversion. Amounts are rounded at conversion by the pair's option (D25); a dropped fraction (`rounding_residual`, signed) stays client money in the balance and pays out when it reaches a unit, an amount added by rounding up is NPL's cost. NPL's margin is captured at conversion and is never again at risk from a return or a forward.
- **Balance conversion** is NPL converting a client's held balance (5K SGD → USDT to fund a 15K USDT payout): Finance-initiated, Management-approved, no sender, receiver or collection, same rate machinery, markup by `FEE_DECISION`. It names the receiver group it serves; `ENTITLEMENT_REATTRIBUTION` moves that group's entitlement to the new currency, and any excess over the group's entitlement is booked as an approved offset.
- **Reroute** is the only way partner or partner entity change after quote. Forwarding is `forward_full` or `forward_and_recover_dues`: NPL nets what the alternate partner owes it (`EARNINGS_RECEIVABLE`) against the funds forwarded, within the outstanding receivable, with the partner's agreement recorded. Post-collection outcome is via the own wallet or a refund to the sender.
- One settlement model: `DISBURSEMENT` drains one partner's holding into one or many `DISBURSEMENT_LINES`, rounded as the pair says. Under `disburse_policy` to `_zero` the balance after release is less than one unit; under `hold_allowed` the balance may rest at the partner within the project's exposure cap.
- A line paying a counterparty receiver whose group differs from the paying leg's attributed group is an offset, created by the same action, with Management approval and tighter aging. A line paying a `sender_return` or `partner_transit` receiver is never an offset: those receivers have no group, and the line debits the group of the leg it names in `deal_id` (D13, D17).
- **Rejected payout.** When the receiving or intermediary bank rejects a transfer, `DISBURSEMENT_RETURN` records the returned amount, bank fee and reason; the line's outcome becomes returned, a reversing ledger transaction puts the money back into the client's balance and the paying leg's group entitlement, and `BANK_FEE_EVENT` books the bounce fee. The money then stays in balance for later payouts, or funds a return leg — attributed to the same group — that converts back and pays the sender's return receiver.
- **Fee decisions.** `FEE_DECISION` is a one-off, approved decision on a single subject: markup applied or waived on a return leg or balance conversion, a network-fee override on a refund, a bank-fee treatment override on one settlement. Standing rules stay in `FEE_STRUCTURE` and `FEE_OVERRIDE`.
- `CONFIRMATION` is full or short; a short confirmation opens a `SHORTFALL` topped up by a later disbursement or an explicit separate payout — cash found short at hand-over included. Rounding differences no longer land here. `REFERRAL_ACCRUAL` and `PARTNER_REBATE_ACCRUAL` accrue per deal under their rules; a `party_retains` referral accrual is a receivable from the party, reconciled monthly like a rebate (D26); rebates are reconciled monthly against the partner's statement, which covers NPLify deals only (D27).
- **Pricing stamps (v5.3).** A leg stamps the override and the base tier that priced it; a leg that crosses a tier threshold holds two `DEAL_FEE_TIER` portions and a blended fee (D20, D21). The deal group stamps the displayed fee and source rate the quote printed (D22). On a pair that needs a bank lock the leg records `rate_locked_at` and `conversion_due_date`: locked before the partner's cutoff it converts the same day, otherwise the next business day at that day's version; the sender's figures are write-once, NPL absorbs the difference as a cutoff-timing variance, and a conversion whose next-day rate breaches the margin is deferred with a reason. The client balance view shows the locked leg's converted value as pending from the lock; a void removes it (D29).
- **Invoices.** `INVOICE` is an obligation in its own currency — a vendor's EUR invoice a reseller customer pays in THB or USDT, in parts, over days. A deal group names the invoice it pays (advance, then

balance); each payout line records how much of the obligation it discharged and the market snapshot used; `INVOICE_BALANCE` is the open remainder (D24).

- **Cash and crypto parts.** A cash part carries the token (the local-currency bill whose serial identifies the agent), the agent's contact and the partner's count; the city is on the pair, which is why the rate, the fee and the partner pair are already the city's (D30). A crypto part carries the sender's declared address and the screening that admitted it, unless the project skips up-front screening (D31).
- **Loss events.** `LOSS_EVENT` records money lost after it was verified or released — a frozen local account, cash short that cannot be recovered, a wrong-chain transfer — on either side, by any method, for a partial or full amount, with the loss split across sender, NPL, partner and client; Management approves the split, one reversing transaction posts it, and a collection-side event moves the part to reversed (D32).

4 · Ledger & controls domain



Notes

- Client due balances are held as one account per project × partner × currency (CL.<PARTNER>.<CCY>.DUE); group entitlement is a view, not an account. holder_type never includes a settlement rail or narrative entity.
- **Ownership flips client → company at one event only: margin recognition at conversion.** NPL never advances money to a receiver, and withholding on a reroute is not a separate event because the margin is always captured by whichever partner converts.
- Recover dues on a reroute posts own-wallet cash against the earnings receivable at the partner: company money on both sides, no income.
- Transaction sources: Collection, Conversion, BalanceConversion, Disbursement, DisbursementReturn, Confirmation, BankFeeEvent, Reroute, Referral, Rebate, LossEvent, Adjustment. Account purposes include PAYABLE, REFERRAL_PAYABLE, SHARE_RECEIVABLE, REBATE_RECEIVABLE, EARN_GROSS, EARN_REBATE, EXP_ROUNDING and EXP_LOSS; ADVANCE does not exist. Cost components include referral, share, rebate, rounding, fx_timing, cutoff_timing and loss.
- **Three streams NPL never mixes (v5.3).** Earnings on deals post to EARN_GROSS at conversion; a partner's rebate posts to EARN_REBATE and is never in the base a client party shares (D26, D27); NPL's remainder from a party that holds the converted funds sits in SHARE_RECEIVABLE until the party pays it. The cost of rounding up is EXP_ROUNDING, NPL's share of a loss event is EXP_LOSS; the client's and sender's shares of a loss reduce the client balance, the partner's share is a receivable from the partner (D25, D32). Own-desk deals use the ordinary partner-coded accounts with the own-desk configuration as holder and EXP_PARTNER zero by construction (D23).
- APPROVAL, AUDIT_LOG and RECORD_LOCK are modelled once and apply to all master data. Approver ≠ initiator on every approval; Operations never sees derived economics. Approval actions include refund_to_third_party, cross_group_payout, balance_conversion, recover_dues, enable_own_desk, approve_loss_split and override_screening_reject. cross_group_payout never fires on a line paying a sender_return or partner_transit receiver. Wallet screening itself is an Operations action, not an approval (D31).

5 · Invariants (enforced at write time)

1. Postings in one transaction sum to zero per currency.
2. Ledger tables are append-only; corrections are reversing transactions.
3. No custody in the normal path: CL.OWN.*.HELD may be debited only by a collection on a deal with an approved REROUTE, or on a leg whose PARTNER_CONFIG is an own desk (v5.3, D23 — the NPL-GR game-reseller project); the custody aging view reports both and is the standing control.
4. Disburse to the residual: under disburse_policy = to_zero a DISBURSEMENT pays the balance rounded by the pair's option and leaves only the rounding residual (dust stays client money; a round-up is NPL's cost — D25); under hold_allowed the balance may rest at the partner within the project's exposure cap (Q5, Q8).
5. Priced once, per leg: a leg's pricing fields are write-once after priced_at; a re-quote is a new leg or a new deal group.
6. Partner fixed at quote, per leg: partner_pair_id / partner_entity_id change only through an approved REROUTE. A collection's endpoint must belong to the leg's partner entity (v5.1, D15). Its kind must match the leg's collection_method — wallet for crypto, bank for bank, no endpoint for cash (v5.2, D19). A COLLECTION may reference only an active endpoint; a bank endpoint is active only with rail set (D18).
7. Entitlement conservation: $\Sigma \text{GROUP_ENTITLEMENT}$ over a project's groups = $\Sigma \text{CLIENT_BALANCE}$ over partners, per currency, including balance-conversion re-attributions. Return and hop legs preserve this because they inherit the parent leg's group (D13). Since v5.2 every payout line resolves to a group — the paid receiver's for a counterparty, the paying leg's via deal_id otherwise — so this is checkable mechanically (D17).
8. Destination approval precedes collection: a quote-first deal cannot leave Inquiry unless at least one account in the entitled group has an approved SETTLEMENT_REGISTRATION with the chosen partner.
9. Approver $\neq$ initiator on every APPROVAL; Operations never receives derived-economics fields.
10. Every balance shown anywhere is a sum over postings or deals.
11. A collect-first deal requires an existing SENDER_RECEIVER_ALLOW entry for its sender and group (Q1).
12. A return leg pays the sender_return RECEIVER linked to the parent leg's sender, unless a Management-approved exception with a reason names another approved receiver.
13. The failed forward line is reversed before a return leg opens, so entitlement and balance are correct at the return conversion.
14. A CONVERSION belongs to exactly one owner: a deal leg or a balance conversion.
15. A recovery on a reroute cannot exceed the outstanding EARNINGS_RECEIVABLE at that partner; the alternate partner's expected conversion is computed on the net amount forwarded.
16. Ownership flips client $\rightarrow$ company at one event only: margin recognition at conversion. NPL never advances money to a receiver (Q13).
17. Offsets arise only when a counterparty receiver is paid from a leg attributed to a different group, and carry Management approval and tighter aging. A project with one group never has an offset; within a group nothing is tracked beyond an informational tally (v5.0).
18. A RECEIVER has a group_id if and only if kind = counterparty. Forward legs and the sender allow list target counterparty groups and receivers only. A return leg inherits receiver_group_id from its

- parent leg; leg n of a route inherits it from leg $n-1$ (v5.0, D13).
19. A DISBURSEMENT_LINE paying a group-less receiver (sender_return, partner_transit) carries deal_id; the paying leg's group is a stored fact, never inferred (v5.2, D17).
 20. A FEE_OVERRIDE with its own split has both share columns set and they sum to its pct; with neither set the structure's proportion applies. The leg stamps the override that priced it (v5.3, D20).
 21. The base tier stamped on a leg is the tier with the highest min_monthly_volume not exceeding the project's calendar-month volume at quote (PROJECT_MONTH_VOLUME); a leg that takes the month past a threshold is split into two DEAL_FEE_TIER portions whose amounts sum to amount_in, and its fee parts are the blended sum (v5.3, D21).
 22. Displayed figures are derived, never typed: $\text{displayed_source_rate} = \text{sender_rate} \div (1 - \text{displayed_fee_pct})$, stamped at quote; the true rates and fee parts are stored beside them and Operations sees the displayed pair only (v5.3, D22).
 23. Σ obligation_discharged over an invoice's confirmed lines never exceeds its amount; a line that names an invoice names a deal_id whose deal group is on that invoice (v5.3, D24).
 24. Rounding is the pair's configured option — amount_rounding / rounding_unit on payouts, cash_rounding / cash_rounding_unit on the cash amount collected; the residual is signed — a dropped fraction stays client money in the balance, an amount added by rounding up is NPL's cost posted to EXP_ROUNDING — so every rounding difference has an owner (v5.3, D25).
 25. Σ pct of the share_of_net_markup rules active on one leg $\leq 100\%$; the share base is the leg's fee parts less its partner cost and never includes a partner rebate. A party_retains accrual is a receivable from the party, reconciled monthly (v5.3, D26).
 26. Rebate income posts only to EARN_REBATE, never to EARN_GROSS, and every rebate accrual names the NPLify deal it arises on; the partner statement is reconciled against those deals alone (v5.3, D27).
 27. A partner rate version stores the rate as quoted and its direction; the normalised rate is derived ($1 \div \text{quoted}$ when the directions differ, at the pair's precision) and is subject to the breach check like any other, so a direction slip freezes the pair instead of pricing a deal (v5.3, D28).
 28. On a pair that needs a bank lock, a leg's conversion_due_date is the lock date when rate_locked_at precedes the partner's cutoff and the next business day otherwise; Converting may not start before it. The sender's figures are write-once (invariant 5); the difference between the stamped partner rate and the rate actually used posts to VARIANCE with component cutoff_timing — NPL absorbs it (v5.3, D29).
 29. A currency pair's cash side names its CASH_LOCATION and a leg's collection_method is cash if and only if its pair's from side does (a cash payout if and only if the to side does); partner pairs, fee structures and rate versions are therefore per city by construction. A cash part is verified only after the partner's count (counted_at) (v5.3, D30, amended 9 October).
 30. A crypto collection names its sending address and, unless the project skips up-front screening, the screening that admitted it; a screening decided accept_alternate may land only on an endpoint whose risk_band is elevated, and no collection exists for a decision of reject unless Finance overrides it (v5.3, D31).
 31. A single_use endpoint or account is referenced by at most one collection or payout line, for the deal it was reserved for, and retires when that reference is verified or confirmed. A LOSS_EVENT's shares sum to its amount, which never exceeds the amount verified or released; a collection-side event moves the part to reversed and the client balance never goes below zero (v5.3, D32).

32. A pair whose two currencies are the same is converted at rate 1 like any other pair, so the sender and receiver fee parts are captured at conversion and the balance arithmetic is unchanged (v5.3, D33).

6 · What changed from v4.0

Draft v5.3 (9 October 2026) applies the thirteen fee-practice gaps found between NPL's fee table, Sud's workbook answers and the Project Context / Payment methods tabs and ERD v5.2 — written up as proposals A–M in "Model Gaps & Proposals v1.0" and confirmed by NPL on 8–9 October — plus the same-currency rule, as decisions D20–D33. Six tables are added (FEE_TIER, DEAL_FEE_TIER, INVOICE, WALLET_SCREENING, LOSS_EVENT, CASH_LOCATION) and two views (INVOICE_BALANCE, PROJECT_MONTH_VOLUME): 60 tables and 6 views. Earlier: v5.0 / v5.1 (7 October) removed the return group (D13), gave accounts explicit wallet fields (D14) and restructured the partner's collection side (D15); v5.2 (8 October 2026) added D16–D19.

v5.3 — the fee-practice decisions (9 October 2026)

D20 (A) — A per-sender override carries its own split

Decision. FEE_OVERRIDE gains sender_share_pct and receiver_share_pct; both blank means the structure's proportion, both set means they sum to pct. DEAL stamps fee_override_id.

Rejected alternative. Keeping the override as a total only, with the structure's proportion.

Why. Evo's selected senders pay 1.0 % while Evo still pays its 0.6 %; a proportional override would charge 1.12 % and 0.48 %. NPL confirmed the receiver part is fixed for every sender.

D21 (B) — Fees by monthly volume

Decision. FEE_TIER under a fee structure (one tier at volume 0); the tier is chosen at quote from the project's calendar-month volume (PROJECT_MONTH_VOLUME) and stamped with the volume that justified it. A leg that takes the month past a threshold is split: DEAL_FEE_TIER holds the portion below the threshold at the lower tier and the portion above at the next, and the leg's fee parts are the blended sum.

Rejected alternative. A rolling 30-day window; switching the whole leg to the new tier; switching from the next leg only.

Why. GDC pays 2.5 % below USD 500,000 a month and 2.25 % above, switched by hand today. NPL: calendar month; the crossing transaction itself attributes the part above the threshold to the lower tier.

D22 (C) — What the quote prints

Decision. FEE_STRUCTURE.displayed_fee_basis (actual / sender_share_on_partner_rate / fixed) and displayed_fee_pct; DEAL_GROUP stamps displayed_fee_pct and displayed_source_rate = sender_rate ÷ (1 – displayed fee) so a re-printed quote matches. Displayed figures are derived, never typed; Operations sees the displayed pair, Finance both.

Rejected alternative. Leaving the recomputation of the 'market rate' to Operations by hand.

Why. BF quotes via Ali show 'fee 1 %' while the recorded parts add to 1.4 % (Ali's 1 % inside its rate plus NPL's 0.4 %). NPL: a BF-only rule, to be cleared when the project moves to partner rate plus markup. The Raeen sub-account quote uses the same recomputation.

D23 (D) — NPL-GR as an own-desk project

Decision. PARTNER_CONFIG.is_own_desk; an own wallet is a COLLECTION_RECEIVING_ENDPOINT under it (own_wallet_id); own-desk pairs are priced market plus NPL's markup with generated rate versions; invariant 3 restated: client money may rest in NPL's wallet under an approved reroute or in an own-desk

project, and the custody view reports both. Enabling an own desk is a Management-approved configuration change (`enable_own_desk`).

Rejected alternative. Keeping the no-custody principle intact and running the game-reseller flows outside NPLify.

Why. The reseller products (Bala, Fair, 66 Group, TripleSeven, Alpha Plus, 27 Group via THB cash) collect into NPL's own wallet and pay vendors from it on later days; there is no outside partner for the conversion. NPL's framing: NPL-GR, a business unit of NPL, is the client. Management confirmed running it inside the system.

D24 (E) — Invoices paid in parts

Decision. `INVOICE` owned by the project, naming the vendor; `DEAL_GROUP.invoice_id`; `DISBURSEMENT_LINE` gains `invoice_id`, `obligation_discharged`, `discharge_rate` and the market snapshot used; `INVOICE_BALANCE` = amount – Σ discharged over confirmed lines. The EUR difference between what was collected and what was discharged posts to variance with component `fx_timing`.

Rejected alternative. Tracking the obligation in a free-text client reference.

Why. A vendor's EUR invoice is settled in USDT in instalments at different days' rates (EUR/USDT 0.8708 at collection, 0.87527 at the last payment); the open balance changes with every instalment. NPL: wanted; vendors are paid on different days and the forex difference is accepted within the 2 % markup.

D25 (F) — A rounding menu per pair

Decision. `CURRENCY_PAIR.amount_rounding` grows from one value to `truncate_unit / nearest_unit / round_up_unit / nearest_n / round_up_n` with `rounding_unit`; `CONVERSION.rounding_residual` is signed — a dropped fraction stays client money, an amount added by rounding up is NPL's cost in `EXP_ROUNDING`. Cash collections use the same menu on the collected amount (`cash_rounding` with `cash_rounding_unit`).

Rejected alternative. Truncation only, with the other practices left outside the system.

Why. Live quotes round THB and INR to the nearest 1,000 and round a reseller customer's USDT up. NPL: round up to the nearest 1,000 for Asian currencies, nearest unit for USD, SGD, HKD; round-up on USDT is deliberate.

D26 (G) — Markup shared with a client party

Decision. `REFERRAL_RULE` basis `share_of_net_markup`: the base is the leg's fee parts less its partner cost, on amount $\times$ market, rebates never included; `party_kind` introducer / client_party; `settlement_mode` `npl_pays` / `party_retains`; optional scope by fee structure; several rules per project with Σ shares ≤ 100 %. `REFERRAL_ACCRUAL` gains direction, period and `received_amount`; a `party_retains` accrual is a receivable (`SHARE_RECEIVABLE`) reconciled monthly.

Rejected alternative. A fixed percentage per party, reconfigured when the route changes; NPL always paying the share out.

Why. Raeen's downlines fund sub-accounts at 4 %; whatever is left after the partner's fee is split equally — 1.5 % each via Ali, 1.6 % each via Aquanow — Raeen keeps the converted EUR in its own Aquanow account and pays NPL monthly; direct downlines split 0.2 % 75 / 25. NPL: future deals may have several parties with different shares; the partner's rebate is NPL's alone and not visible to the party.

D27 (H) — Rebates as their own stream

Decision. Rebate income posts to `EARN_REBATE`, never `EARN_GROSS`. `PARTNER_REBATE_RULE` gains basis (`pct_of_amount / pct_of_partner_fee`) and `pct`, retires the old `pct_of_partner_fee` column (amended 9

October), and gains an optional scope by sender or fee structure. PARTNER_REBATE_ACCRUAL keeps deal_id required and gains statement_ref.

Rejected alternative. Accruals without a deal for clients NPL referred but does not service.

Why. NPL: the partner statement that NPLify reconciles covers only deals NPLify handled — customers working with Jeton or Aquanow directly are out of scope. Jeton pays 0.4 % of the amount on a 1 % or 1.25 % fee and 0.3 % on Novi's 0.9 %, so the rebate is not a fixed share of the fee. Management reads earnings and rebates as two numbers.

D28 (I) — Partner rates as quoted

Decision. PARTNER_RATE_VERSION gains quoted_rate and quoted_direction; rate is derived ($1 \div$ quoted when the directions differ, at the pair's precision) and goes through the breach check like any rate.

Rejected alternative. A single normalised rate typed by Operations.

Why. Ali sometimes states USDT → EUR as 1.1738 (EUR-per-USDT the other way round); the hand inversion is a 38 % error waiting to happen, and the audit trail should show the number the partner sent.

D29 (J) — Bank cutoff, rate lock, next-day conversion

Decision. PARTNER.bank_cutoff_time / cutoff_timezone (amended 9 October: on PARTNER, not PARTNER_CONFIG — the cutoff is the partner's, whichever project uses it); PARTNER_PAIR.needs_bank_lock; DEAL and BALANCE_CONVERSION carry rate_locked_at and conversion_due_date (the lock date before the cutoff, else the next business day; Converting may not start earlier) and a conversion_deferred_reason. The client balance view shows a locked leg's converted value as pending from the lock; a void removes it. The difference between the stamped partner rate and the one used posts to variance with component cutoff_timing.

Rejected alternative. Treating the cutoff as part of quote validity; re-quoting the sender the next day.

Why. Evo via Ali: the rate is compared in the morning; locked before 14:30 GMT+8 it converts the same day, otherwise the money is collected on the morning's figures and converted the next business day. AK's USD and SGD balances are banking-ready and wait too. NPL: NPL absorbs the difference; if Ali's next-day rate breaches the margin, ask for a better rate or wait a day; today's balances should show the pending converted value.

D30 (K) — Cash by token, rates per city

Decision. CASH_LOCATION (currency, city, country) named by the cash side of a CURRENCY_PAIR — from_cash_location_id / to_cash_location_id — so INR (Cash-Delhi) → USDT and INR (Cash-Mumbai) → USDT are two pairs and partner pairs, fee structures and rate days are per city by construction (amended 9 October; the first draft put the city on the rate version and the deal); COLLECTION gains token_ref, agent_contact_ref and counted_at, with cash states token_issued → handed_over → counted → verified; DISBURSEMENT_LINE.token_ref for cash payouts.

Rejected alternative. Free-text cash instructions on the deal and one rate per currency; a city column on each rate version and deal (the first v5.3 draft).

Why. A cash collection or settlement is arranged by token — a local-currency bill whose serial number identifies the agent — closed by a photo of the token and the partner's count; INR in Delhi and Mumbai, or USD cash in Bangkok and Singapore, are different agents at different rates.

D31 (L) — Wallet screening

Decision. WALLET_SCREENING per address and day for sending and receiving wallets (result, decision, who, when, evidence); COLLECTION gains sending_address, wallet_screening_id and screening_skipped;

`COLLECTION_RECEIVING_ENDPOINT.risk_band`; `SETTLEMENT_REGISTRATION.wallet_screening_id`. A screening decided `accept_alternate` lands only on an elevated-band endpoint; a reject is overridden only by Finance.

Rejected alternative. A static risk tier on the sender.

Why. NPL screens the sender's declared wallet before giving out the partner's receiving wallet, routes elevated-risk wallets to alternate wallets, verifies the hash on chain, and screens receivers' wallets before settlement. NPL: screening is a transaction-coordination activity, so it is an Operations action and Operations sees the result. Not every Evo sender accepts up-front screening.

D32 (M) — Single-use accounts and loss events

Decision. `COLLECTION_RECEIVING_ENDPOINT` and `RECEIVING_ENTITY_ACCOUNT` gain usage `single_use` with `reserved_for_deal_id` (and an expected amount on the endpoint); `LOSS_EVENT` on either side, by any method, for a partial or full amount, with the loss split across sender, NPL, partner and client (shares sum to the amount, Management approves); one reversing transaction (source `LossEvent`); `EXP_LOSS`; `COLLECTION`. state reversed. Cash found short at hand-over is a short confirmation topped up at the next settlement or by a separate payout, and a loss event only when unrecoverable.

Rejected alternative. A collection-only reversal borne by the sender in full.

Why. Local deposits use accounts the partner lends for one transaction; a frozen account can lose the funds. NPL: the loss can be partial, can be shared between NPL and the partner as well as the sender, can happen on the settlement side and by other methods, and single-use local accounts are used for settlement too.

D33 — Same-currency pass-through stays in scope

Decision. A `CURRENCY_PAIR` may have the same currency on both sides; the leg converts at rate 1 so the sender and receiver fee parts are captured at conversion like on any pair.

Rejected alternative. Dropping the dormant Evolution bank-transfer products.

Why. NPL: the fee is still applied because the currency is collected and then settled separately.

Amendments of 9 October (same draft)

D27. `pct_of_partner_fee` is retired; `basis + pct` carry the rebate rule. **D29.** The bank cutoff moves to `PARTNER`: it is the partner's clock, like its rate expiry and holiday calendar, whichever project uses it. **D30.** The cash city is a side of the currency pair, not a column on each rate version and deal: `CASH_LOCATION` is named by `CURRENCY_PAIR.from_cash_location_id / to_cash_location_id`, so `INR (Cash-Delhi) → USDT` and `INR (Cash-Mumbai) → USDT` are two pairs and everything that hangs off a pair — partner pairs, fee structures, rate days — is per city by construction; bank and crypto sides carry no location. Stale wording from before D23, D25 and D26 was corrected in the notes, walkthroughs, invariant 4 and the FRS reconciliation.

New tables (v5.3)

Table	Holds
<code>FEE_TIER</code>	A volume tier of a fee structure: minimum calendar-month volume, currency, pct, optional split (D21).
<code>DEAL_FEE_TIER</code>	The tier portions of one leg — two when the leg crosses a threshold; amounts sum to <code>amount_in</code> (D21).
<code>INVOICE</code>	A vendor obligation in its own currency, owned by the project, paid by one or more deal groups and discharged line by line (D24).

Table	Holds
WALLET_SCREENING	A screening of one wallet address on one day — sender or receiver side — with result, decision, who and evidence (D31).
LOSS_EVENT	Money lost after verification or release, on either side, partial or full, with the loss split across sender, NPL, partner and client (D32).
CASH_LOCATION	A city where a currency is handled as cash; named by the cash side of a currency pair (D30, amended 9 October).

Changed tables (v5.3)

Table	Change
FEE_OVERRIDE	+ sender_share_pct, receiver_share_pct (D20).
FEE_STRUCTURE	+ displayed_fee_basis, displayed_fee_pct (D22); owns FEE_TIER (D21); may scope a referral or rebate rule (D26, D27).
DEAL	+ fee_override_id, fee_tier_id, tier_volume_basis (D20, D21); + rate_locked_at, conversion_due_date, conversion_deferred_reason (D29); collection_method is cash iff the pair's from side names a city (D30).
DEAL_GROUP	+ displayed_fee_pct, displayed_source_rate (D22); + invoice_id (D24).
PARTNER	+ bank_cutoff_time, cutoff_timezone (D29, amended 9 October: moved here from PARTNER_CONFIG).
PARTNER_CONFIG	+ is_own_desk (D23).
PARTNER_PAIR	+ needs_bank_lock (D29).
COLLECTION_RECEIVING_ENDPOINT	+ own_wallet_id (D23); + risk_band (D31); + usage, reserved_for_deal_id, expected_amount (D32).
RECEIVING_ENTITY_ACCOUNT	+ usage, reserved_for_deal_id (D32).
CURRENCY_PAIR	amount_rounding becomes a menu; + rounding_unit; + cash_rounding, the same menu for the collected cash amount (D25); + from_cash_location_id, to_cash_location_id (D30); from_currency may equal to_currency (D33).
CONVERSION	rounding_residual signed (D25); rate 1 on a same-currency pair (D33).
REFERRAL_RULE	basis + share_of_net_markup; + fee_structure_id, party_kind, settlement_mode (D26).
REFERRAL_ACCRUAL	+ direction, period, received_amount; states accrued / reconciled / settled (D26).
PARTNER_REBATE_RULE	+ basis, pct, fee_structure_id, sender_id; – pct_of_partner_fee (D27).
PARTNER_REBATE_ACCRUAL	+ statement_ref; deal_id stays required (D27).
PARTNER_RATE_VERSION	+ quoted_rate, quoted_direction (D28); rate derived.
BALANCE_CONVERSION	+ rate_locked_at, conversion_due_date (D29).
COLLECTION	+ sending_address, wallet_screening_id, screening_skipped (D31); + token_ref, agent_contact_ref, counted_at (D30); state + reversed, cash sub-states (D30, D32).
DISBURSEMENT_LINE	+ invoice_id, obligation_discharged, discharge_rate, discharge_market_rate_id (D24); + token_ref (D30).
SETTLEMENT_REGISTRATION	+ wallet_screening_id (D31).

Table	Change
LEDGER_ACCOUNT	purposes + EARN_REBATE (D27), SHARE_RECEIVABLE (D26), EXP_ROUNDING (D25), EXP_LOSS (D32).
LEDGER_TRANSACTION	source types + LossEvent (D32).
LEDGER_POSTING	cost components + rebate, share, fx_timing, cutoff_timing, loss.
APPROVAL	actions + enable_own_desk (D23), approve_loss_split (D32), override_screening_reject (D31).

Views and invariants (v5.3)

Item	Change
CLIENT_BALANCE	+ pending: the expected_out of rate-locked legs not yet collected or converted; removed on void; never a posting (D29).
CUSTODY	Includes own-desk client balances with their age (D23).
INVOICE_BALANCE	New: invoice amount – Σ obligation_discharged over confirmed lines (D24).
PROJECT_MONTH_VOLUME	New: Σ amount_in of the project's forward legs in the calendar month, in the tier currency; read at quote (D21).
Invariant 3	Restated: custody is allowed under an approved reroute or in an own-desk project; the custody view reports both (D23).
Invariants 20–32	New, one per decision D20–D33 (invariant 32 covers D33): override split, tier choice and split, displayed figures derived, invoice discharge bounded, signed rounding residual with an owner, share base and Σ shares, rebate stream and deal-bound accruals, rate derived from the quoted rate, conversion due date and cutoff variance, cash city on the pair and cash count, screening $\leftrightarrow$ endpoint band, single-use references and loss shares, same-currency conversion at rate 1.

Dependent documents — not yet regenerated against v5.3

Document	What v5.3 changes in it
Ledger Posting Design v1.1 → v1.2	Four new worked patterns: party-retains share (SHARE_RECEIVABLE), rebate as its own stream (EARN_REBATE), partial loss with a split (LossEvent, EXP_LOSS), own-desk deal (NPL-GR); signed rounding residual and EXP_ROUNDING; fx_timing and cutoff_timing variances.
Lifecycle & Approvals v1.1 → v1.2	Rate-locked and awaiting-cutoff sub-states with the deferral; loss event path with Management approval; cash collection steps (token → hand-over → count); screening before Collecting as an Operations action; enable_own_desk.
Roles & Visibility Matrix v1.0 → v1.1	Screening fields in the Operations group; rebate, party share and loss split in the economics group; a party-facing statement shows the share only, never the rebate.
Calculation Specification v1.0 → v1.1	Tier split and blended fee; displayed source rate; rate-direction normalisation; rounding menu; same-currency conversion at rate 1; acceptance vectors from NPL's real figures (market 0.86050 / Ali 1 ÷ 1.1738; GDC 86,594; Raeen 41,520.17; THB 17,000,000; USDT 2,598 → 2,600).
Configuration Schema v1.0 → v1.1	Every new column and value set; the NPL-GR own-desk project; rebate rule basis and scope; cutoff times; cash locations; single-use endpoints.
ERD Reading Guide v1.0 → v1.1	The new boxes on 1a, 1b, 3a and 3b, the split of the deal diagram into 3a and 3c, and the decisions behind them.

Document	What v5.3 changes in it
Fee Outlines v0.2 → v0.3	Pink cells resolved (Ad Hoc INR 7 % total; Raeen 3.2 %; INR rounding); 'needs a system change' notes become 'configured in v5.3'; the tier split shown on pattern 5.
Model Gaps & Proposals v1.0 → v1.1	The corrections of 9 October (H simplified, J pending balance, L Operations, M generalised, B split) and the status 'applied in v5.3' on all thirteen.
ERD Review — Decision Record	D20–D33 recorded (9 October 2026).
data/erd.v5.json, interactive map, knowledge base	The data file is at 5.3 with this draft; the map and the knowledge base still read 5.2.

Earlier revisions — v5.0 to v5.2

D13 — No return group

Decision. sender_return and partner_transit receivers belong to no group. Attribution of a return or hop leg is inherited from its parent leg. Offsets are defined between a leg's attributed group and the group of a counterparty receiver.

Rejected alternative. A dedicated group of kind = return per project holding the sender_return receivers (ERD v4.0 §6; config schema rule "one default and one return group").

Why. Groups exist for entitlement and offset accounting; refund exclusion is a receiver-kind rule, not an entitlement rule, and RECEIVER.kind already enforces it. The return group (a) made every project have at least two groups, so the Understanding document's "a project with a single group never has an offset" was false by construction; (b) classified every refund as a cross-group payout under the v4.0 offset definition, triggering Management approval and offset aging; (c) left the return group's entitlement negative after each refund, breaking invariant 7, unless a reattribution path was added. Removing it restores the Understanding document's §3.3 and §6.3 wording without change. The return group entered during the v4.0 write-up; decision D3 of the review record relied on it and is amended there.

Changed tables

Table	Change
RECEIVER_GROUP	– kind. A group is a group; exactly one is _default per project (D13).
RECEIVER	group_id required when kind = counterparty, null otherwise. sender_return and partner_transit receivers never appear on forward deals or allow lists (validation on kind) (D13).
DEAL	receiver_group_id: set at quote on the first forward leg; a return leg inherits it from its parent leg, leg n of a route from leg n–1 (D13).
DISBURSEMENT_LINE	A line paying a group-less receiver debits the paying leg's group and is never an offset (D13).
RECEIVING_ENTITY_ACCOUNT	+ account_kind (bank / wallet); details_ref → bank_details_ref (bank accounts only); + network, + address (wallets only). Write-time rule: a bank account has bank_details_ref and a bank rail; a wallet has network and address and a crypto rail matching the network (D14).

D14 — Explicit wallet fields on receiving-entity accounts. Rejected: keeping wallet details inside the opaque details_ref. Why: the collection side already stored network and address explicitly; the payout side did not, so addresses could not be validated, checked against the rail, or queried across partners. Bank details stay external by reference, as before.

D15 — Partner entity and collection endpoint

Decision. FACILITATING_ENTITY is renamed PARTNER_ENTITY: the partner's legal vehicle whose name appears on paperwork, the mirror of RECEIVING_ENTITY on the client side. COLLECTION_RECEIVING_ENTITY (bank-in) and WALLET (crypto-in) merge into one COLLECTION_RECEIVING_ENDPOINT table owned by a partner entity, with kind = bank / wallet and the field shape of RECEIVING_ENTITY_ACCOUNT (rail required only on active bank endpoints, see D18). COLLECTION points at one endpoint (collection_receiving_endpoint_id, null for cash) instead of the hand-rolled endpoint_type + endpoint_id pair. The rename is pending NPL confirmation; the FRS term "Facilitating Entity" stays in Appendix A.1.

Rejected alternative. Keeping two endpoint tables, one with a legal owner (wallets under a vehicle) and one without (bank accounts directly under the partner config).

Why. A bank account is always in some legal entity's name, so the v4.0 split left the schema unable to say which vehicle owns a bank-in account, while every wallet knew its owner. One endpoint table under the vehicle fixes the asymmetry, lets the system check that the account a sender paid into belongs to the vehicle stamped on the deal, removes a polymorphic pair from COLLECTION, and makes the partner side read exactly like the client side: config → entity → endpoint.

Table	Change
PARTNER_ENTITY	renamed from FACILITATING_ENTITY; – bank_details_ref (moved to its endpoints).
COLLECTION_RECEIVING_ENDPOINT	new; replaces COLLECTION_RECEIVING_ENTITY and WALLET. partner_entity_id, kind (bank / wallet), currency, bank_details_ref (bank), network + address (wallet), status.
COLLECTION	endpoint_type + endpoint_id → collection_receiving_endpoint_id (null for cash).
DEAL	facilitating_entity_id → partner_entity_id.
Invariant 6	adds: a collection's endpoint must belong to the leg's partner entity.

v5.2 — decisions of the review of the v5.1 draft (8 October 2026)

D16 — Registration payout details follow the account's kind

Decision. SETTLEMENT_REGISTRATION gains network and address (nullable). bank_details_ref ("as paid by this partner") is set only when the registered account's account_kind is bank; for a wallet account the registration carries the network and address as the partner registered them, and the bank field stays empty. The registration remains the partner's view of the destination; RECEIVING_ENTITY_ACCOUNT remains NPL's.

Rejected alternative. Keeping a bank-only field on wallet registrations (the v5.1 shape).

Why. After D14 and D15 every other destination record is kind-aware; a wallet registration had no field that fitted, so an approved wallet destination either carried no details or carried an address in a field named for bank details. Storing the partner's copy also lets the system compare NPL's record with the partner's before paying to a wallet, where a wrong address is unrecoverable.

D17 — DISBURSEMENT_LINE.deal_id

Decision. DISBURSEMENT_LINE gains deal_id, a nullable foreign key to DEAL: required when the paid receiver is group-less (sender_return, partner_transit), optional on counterparty lines. New invariant 19. GROUP_ENTITLEMENT reads the paying leg's group from it.

Rejected alternative. Leaving the rule in prose ("a payout to a group-less receiver debits the paying leg's group").

Why. A DISBURSEMENT pools many deals, and neither it nor its lines named one; the D13 rule and the view that depends on it could not be computed from stored columns, and invariant 7 could not be checked mechanically. The same column will carry the per-line discharge facts of the scenario in which a EUR invoice is paid in USDT in parts.

D18 — Nullable rail on COLLECTION_RECEIVING_ENDPOINT

Decision. The endpoint gains rail, nullable. Null means "not yet known", never "does not matter": a bank endpoint cannot become active — and no COLLECTION or payment instruction may reference it — until rail

is set; a wallet endpoint may leave rail null (its network already names the road), and when rail is set on a wallet it must match network (mirrors D14).

Rejected alternative. A NOT NULL rail, which forces a guessed value when a partner sends incomplete details; or no rail at all (the v5.1 shape), which made the "same field shape as RECEIVING_ENTITY_ACCOUNT" claim inexact.

Why. The road matters on the collection side as much as on the payout side: the same EUR account reachable by SEPA or SWIFT needs different instructions and arrives with different fees; local rails carry transaction limits (UPI caps, RTGS minimums); and a wrong rail recorded with confidence causes the failures a blank cannot.

D19 — Endpoint kind must match the leg's collection method

Decision. A write-time rule, added to invariant 6: a collection endpoint's kind must match the leg's collection_method — wallet for crypto-in, bank for bank-in, and cash-in has no endpoint.

Rejected alternative. None; the rule was missing.

Why. After D15 the fact "how does the money arrive" lives in two places — DEAL.collection_method and the endpoint's kind — and nothing stopped them disagreeing silently.

Changed tables (v5.2)

Table	Change
SETTLEMENT_REGISTRATION	+ network, + address (wallet accounts only); bank_details_ref only when the account's kind is bank (D16).
DISBURSEMENT_LINE	+ deal_id — nullable FK to DEAL; required when the paid receiver is group-less, optional on counterparty lines (D17).
COLLECTION_RECEIVING_ENDPOINT	+ rail — nullable; required before a bank endpoint becomes active; on a wallet, must match network when set (D18).
COLLECTION	No column change: may reference only an active endpoint whose kind matches the leg's collection_method (D18, D19).

Views and invariants

Item	Change
GROUP_ENTITLEMENT	Sums client_net over converted legs of both types; a payout line debits the paid receiver's group (counterparty) or the group of the leg named by its deal_id (group-less receivers) — computable from stored columns (D13, D17).
OFFSET	Entitlement — paid per group, arising only when a counterparty receiver is paid from a leg attributed to another group.
Invariant 6	Adds: a collection's endpoint must belong to the leg's partner entity (D15); its kind must match the leg's collection_method (D19); a COLLECTION may reference only an active endpoint, and a bank endpoint is active only with rail set (D18).
Invariant 7	Restated: return and hop legs preserve conservation by inheriting the parent leg's group; since v5.2 every payout line resolves to a group, so the invariant is checkable mechanically (D17).
Invariant 17	Restated: offsets only between a leg's attributed group and a counterparty receiver's group; a single-group project never has an offset.
Invariant 18	group_id iff kind = counterparty; forward legs and the allow list target counterparty groups and receivers only; return and hop legs inherit attribution.

Item	Change
Invariant 19	New: a line paying a group-less receiver carries deal_id; the paying leg's group is a stored fact, never inferred (D17).

Editorial corrections in v5.2

- Invariant 18's note now says the allow list targets counterparty *groups and receivers*, not receivers alone.
- The D13 write-up no longer claims the return group appeared in no decision-record entry: decision D3 of the review record did place sender_return receivers in a return group; the record amends D3 and carries D13–D19.
- The dependent-documents table is refreshed: the data file is already erd.v5.json and the decision record carries D13–D19.
- The third v5.1 change (D15) is dated 8 October 2026; this baseline is an internal New XP document from which the client deliverables are derived.

Dependent documents as updated for v5.2

Document	Change
Configuration Schema Specification v1.0	Replace "one default and one return group" with "exactly one default group"; add "every counterparty receiver names a group; no other kind does". Partner section: entities own endpoints; endpoint kind decides which fields are required; rail nullable with the activation rule (D18); registration fields follow the account's kind (D16). Re-validate the FRS Client 1 example — it should pass with one group.
Lifecycle & Approvals v1.0	cross_group_payout must not fire on return legs or hop legs; endpoint activation gated on rail (D18); collection verification checks the endpoint's kind against the leg's collection_method (D19).
Ledger Posting Design v1.0	No account changes. Confirm the return-leg example posts the payout against the parent leg's group entitlement; payout postings may cite the line's deal_id (D17).
Calculation Specification v1.0	No change.
ERD Review — Decision Record	D13–D19 recorded (8 October 2026).
Understanding document v1.1	Nothing to apply; it never had a return group.
data/erd.v5.json, interactive map, knowledge base	Updated to version 5.2 with this draft (fields, views, invariants, notes, vocabulary).

Carried over from v4.0 unchanged

The 12 new tables, 16 changed tables and 3 removals listed in v4.0 §6 (INTRODUCER, RECEIVER_GROUP, RECEIVING_ENTITY_ACCOUNT, REFERRAL_RULE, PARTNER_REBATE_RULE, DEAL_GROUP, BALANCE_CONVERSION, ENTITLEMENT_REATTRIBUTION, DISBURSEMENT_RETURN, FEE_DECISION, REFERRAL_ACCRUAL, PARTNER_REBATE_ACCRUAL; the removal of RECEIVER_ENTITLEMENT, withhold_earnings and the ADVANCE purpose) all stand. Of the small gaps noted in the engagement record, cash rounding per currency is closed by D25 and the partner's time zone by D29 (cutoff_timezone); EXCEPTION entity, PARTNER_STATEMENT (a header for the monthly rebate statement would sit above PARTNER_REBATE_ACCRUAL.statement_ref), configuration versioning, SENDER_CREDIT.applied_to_deal_id and DEAL_GROUP.estimate_out remain open.

7 · Open questions

1 of the questions carried from the Understanding document remain open; the rest were answered by NPL in the review of the Understanding document or settled in the ERD reviews of 6–9 October 2026 and are reflected in this draft.

Still open

#	Question	Blocks	Status
9	FRS figure reconciliation (1,208 vs 1,225; Mode 1 example)	Golden cases	Sud: Needs clarification. To be discussed.

Answered and reflected in v5.3

#	Question	Answer	In the model
1	Collect-first: must SENDER_RECEIVER_ALLOW pre-exist for attribution; handling of inbound matching no sender	Sud: Yes, sender-receiver mapping should be done in configuration. When the transaction is created, the sender is picked from a list.	v4.0: write-time rule that a collect-first deal needs an existing allow entry; the allow list now targets receiver groups.
2	Withholding-eligibility criteria on rerouted deals	Review (7 Oct): NPL's cut is always captured at conversion, so it is never at risk from a return or a forward. If forwarding happens before conversion, whichever partner converts captures it.	v4.0: the eligibility question dissolves. forwarding_policy is forward_full or forward_and_recover_dues; withhold_earnings is removed. Ownership flips at one event only.
3	Post-collection reroute (funds stuck at a partner): transfer, offset, or refund-and-requote	Sud: The partner can send the funds to NPL (and NPL decides where to send) or the partner can refund the sender.	v4.0: REROUTE.outcome via_own_wallet / refund_sender; a refund is a return leg paying the sender_return receiver.
4	Receiving-entity constraints (currency, rail) narrowing settlement choice	Sud: Receiving entities have specific bank accounts with currency and rail restrictions, for example EUR via SWIFT and EUR via SEPA.	v4.0: RECEIVING_ENTITY_ACCOUNT; registrations and payout lines point at the account.
5	Any deliberate partial-disbursement / reserve policy	Sud: Yes, possible. In some projects the client's funds sit until they are ready to receive them, within an exposure threshold NPL sets by appetite.	v4.0: PROJECT.disburse_policy to_zero / hold_allowed with exposure_ceiling as the cap; invariant 4 restated.

#	Question	Answer	In the model
6	Receiver-side fee timing per project	Sud: Receiver-side fee should be applied at the time of conversion, otherwise the balance will be incorrect.	v4.0: default at <code>_conversion</code> ; <code>CONVERSION.fee_receiver_part</code> ; <code>DISBURSEMENT_LINE.receiver_fee_applied</code> becomes exceptional.
7	Network-fee policy on refunds	Sud: Set at project level with per-transaction override (one 100 USD fee was repaid in the next settlement).	v4.0: <code>PROJECT.network_fee_policy</code> plus <code>FEE_DECISION</code> for the override.
8	Whole-unit conversion rounding rule	Review (7 Oct): fractions are dropped to whole units at conversion, per transaction (30.152 → 30). The dropped fraction stays client money, carried in the balance and paid out when it reaches a whole unit. Cash collections round to the nearest 100 / 500 / 1000 by currency.	v4.0: <code>CONVERSION.rounding_residual</code> ; <code>CURRENCY_PAIR.amount_rounding</code> and <code>cash_rounding_unit</code> ; disburse to less than one unit; <code>SHORTFALL</code> no longer takes rounding. Superseded in v5.3 by D25: a rounding menu per pair, signed residual, <code>EXP_ROUNDING</code> for round-ups.
10	AQN-LT-Sub “settled by offset against transactions outside the system”	Sud: This is NPL’s wallet (pass-through at collection or on the settlement side). AQN is Aquanow, a crypto exchange; LT Sub is the account name.	v4.0: <code>OWN_WALLET.provider</code> and <code>account_label</code> ; pass-through is a two-leg route or a reroute via the own wallet.
11	Reference workbooks per project archetype	Fee Outlines v0.2 (8 October 2026): every line of NPL’s fee table as a fill-in outline with a worked calculation per transaction pattern, answered by Sud with the Project Context and Payment methods tabs.	v5.3: decisions D20–D33 were built from it; the workbook is the acceptance reference for the fee engine.
12	FRS P1 / P3 vs rerouting and multi-partner use — amend FRS or restrict scope	Sud: Check the added explanations (partner types and channels; pass-through and double conversions in §7.1.1). Ask again if still unclear.	v4.0: two-leg routes are deal groups with two legs and a <code>partner_transit_receiver</code> for the hop; FRS principles generalised, scope kept.
13	Advance funding source	Review (7 Oct): never. NPL does not pay a receiver before the client’s money has arrived.	v4.0: <code>LEDGER_ACCOUNT</code> purpose <code>ADVANCE</code> removed; the advance approval and the “advance” ownership-flip event removed.

#	Question	Answer	In the model
14	Review point: entitlement split on a balance conversion	Review (7 Oct): chosen group, not proportional. The conversion is triggered by a specific payout.	v4.0: BALANCE_CONVERSION.receiver_group_id; excess over the group's source-currency entitlement is booked as an approved offset.
15	Review point: refunds to a third party at the sender's request	Review (7 Oct): allowed by exception only. Default is the sender's own return receiver; any other destination needs Management approval with a reason and a pre-registered approved receiver.	v4.0: invariant on return legs; APPROVAL action refund_to_third_party.
16	Review point: recover dues on a reroute	Review (7 Oct): yes, NPL nets what a partner owes it against funds about to be forwarded to that partner.	v4.0: forwarding_policy forward_and_recover_dues, recovered_amount, partner_agreement_ref; postings own wallet cash ← earnings receivable, no income.
17	Review point: legs of a route share one client-facing quote	Review (7 Oct): yes, a deal group. The sender saw one price and one reference; margin across legs is the group quote minus the partner conversions.	v4.0: DEAL_GROUP; every deal belongs to a group, a single-partner deal being a group of one leg.
18	Review point (v5.0): does a refund need its own receiver group?	Review (7 Oct, New XP): No. Refund exclusion is a receiver-kind rule; a dedicated return group made every project multi-group (so "a single-group project never has an offset" was false by construction), classified every refund as a cross-group offset needing Management approval, and left its entitlement negative after each refund, breaking invariant 7.	v5.0 (D13): sender_return and partner_transit receivers are group-less; return and hop legs inherit the parent leg's attribution; offsets are defined only against counterparty groups. New invariant 18.

#	Question	Answer	In the model
19	Review point (v5.1): restructure the partner collection side — rename FACILITATING_ENTITY, merge the endpoints?	Review (8 Oct, New XP): yes. A bank account is always in some legal entity's name, so the v4.0 split could not say which vehicle owned a bank-in account while every wallet knew its owner. The rename to PARTNER_ENTITY is pending NPL confirmation; the FRS keeps "Facilitating Entity".	D15: FACILITATING_ENTITY → PARTNER_ENTITY (bank_details_ref moved to its endpoints); COLLECTION_RECEIVING_ENTITY and WALLET merge into COLLECTION_RECEIVING_ENDPOINT (kind bank / wallet, same shape as RECEIVING_ENTITY_ACCOUNT); COLLECTION points at one endpoint (null for cash); invariant 6 adds that a collection's endpoint must belong to the leg's partner entity.
20	Review point (v5.3): can the model express NPL's fee practice as described in the fee table, Sud's workbook answers and the Project Context / Payment methods tabs (gaps A–M)?	NPL (8–9 Oct): all thirteen confirmed with parameters — receiver part fixed (A); calendar-month tiers, a crossing deal split across tiers (B); displayed 1 % is a BF-only rule (C); run NPL-GR inside NPLify as an own desk (D); invoice tracking wanted (E); rounding menu, round up to 1,000 for Asian currencies (F); markup shared = fee – partner fee, several parties, party holds the funds, rebate NPL's alone (G); rebates reported apart, statement covers NPLify deals only (H); Ali quotes inverted (I); NPL absorbs the next-day rate difference, pending converted value shown from the lock (J); cash by token, rates by city (K); screening is an Operations action (L); losses partial and shared, both sides, any method (M); same-currency pass-through stays in scope.	v5.3, D20–D33: FEE_TIER, DEAL_FEE_TIER, INVOICE, WALLET_SCREENING, LOSS_EVENT; views INVOICE_BALANCE, PROJECT_MONTH_VOLUME; invariants 20–32 and invariant 3 restated; purposes EARN_REBATE, SHARE_RECEIVABLE, EXP_ROUNDING, EXP_LOSS.

Appendix A · FRS reconciliation

A.1 Vocabulary

FRS term	Model entity
Client	PROJECT
Receiving Entity	RECEIVING_ENTITY (member of a counterparty RECEIVER, which belongs to a RECEIVER_GROUP) with its RECEIVING_ENTITY_ACCOUNTs
Sender	SENDER with SENDER_RECEIVER_ALLOW; its refund destination is a group-less sender_return RECEIVER
Partner (PT, Jeton)	PARTNER + PARTNER_CONFIG
Facilitating Entity / Wallet registry	PARTNER_ENTITY with its COLLECTION_RECEIVING_ENDPOINTs (kind = bank / wallet) (D15)
Rate-day versions	RATE_COMPARISON + PARTNER_RATE_VERSION
Settlement	DISBURSEMENT + DISBURSEMENT_LINE + CONFIRMATION (+ DISBURSEMENT_RETURN when rejected)
Shortfall ledger	SHORTFALL + CL.<RECEIVER>.<CCY>.SHORTFALL
PT Comms / H Comms	CO.<PARTNER>.<CCY>.POOL (earnings receivable)
Inter-entity redirection	Offset across receiver groups: a counterparty paid_receiver_id in another group than the paying leg's
Double conversion / pass-through	DEAL_GROUP with two legs (funding_source = prior_leg), hop via a group-less partner_transit RECEIVER
Commission / rebate (Jeton 0.4 %, Aquanow 0.3 %)	PARTNER_REBATE_RULE (basis pct_of_amount or pct_of_partner_fee, scoped by sender or fee structure) + PARTNER_REBATE_ACCRUAL; income in EARN_REBATE (D27)
Markup shared with a customer (Raean sub-accounts, downlines)	REFERRAL_RULE basis share_of_net_markup, party_kind client_party, settlement_mode party_retains; SHARE_RECEIVABLE (D26)
Fee by monthly volume (GDC)	FEE_TIER under FEE_STRUCTURE; DEAL_FEE_TIER portions; PROJECT_MONTH_VOLUME (D21)
Game reseller / NPL-GR	PROJECT with an own-desk PARTNER_CONFIG; OWN_WALLET as COLLECTION_RECEIVING_ENDPOINT; INVOICE paid in parts (D23, D24)
Customer-facing calculation ('market rate', 'fee 1 %')	FEE_STRUCTURE.displayed_fee_basis / pct; DEAL_GROUP.displayed_source_rate (D22)
Token (cash collection / settlement)	COLLECTION.token_ref, agent_contact_ref, counted_at; DISBURSEMENT_LINE.token_ref; CURRENCY_PAIR.from_cash_location_id / to_cash_location_id → CASH_LOCATION (D30)
Bank cutoff (2.30 pm GMT+8)	PARTNER.bank_cutoff_time; PARTNER_PAIR.needs_bank_lock; DEAL.rate_locked_at, conversion_due_date (D29)
Wallet screening	WALLET_SCREENING; COLLECTION.sending_address; COLLECTION_RECEIVING_ENDPOINT.risk_band (D31)
Reversal of collection; cash short	LOSS_EVENT with its split; SHORTFALL for recoverable cash shortages (D32)

A.2 Deviations from the FRS (classified)

FRS statement	Model	Classification
P1 — no custody	No custody except under an approved reroute or in an own-desk project (NPL-GR); OWN_WALLET + REROUTE for contingency, invariant 3 and the custody view as the control	Decided (Q12 answered; D23, 9 October 2026)
P3 — partner books never mix	Accounts per partner never net; alternate partner on reroute; post-collection transfer only if Q3 confirms	Decided (Q12 answered: reroute with recover dues; partner accounts never net)
Strict Client → Entity → Sender, one parent each	RECEIVER_GROUP → RECEIVER → RECEIVING_ENTITY → account; SENDER with a group allow-list; FRS project = one default group	Generalisation per NPL instruction
Beyond grace → re-quote always	late_collection_policy per project; FRS project keeps requote	Per NPL instruction (Sept 2026)
Sender always priced on Kraken	FEE_STRUCTURE.rate_basis; FRS project = market	Generalisation
One settlement per deal, full amount	Accumulate → disburse the balance rounded as the pair says (residual carried), or hold within the exposure cap where the project allows	Generalisation (D25)
Partner fixed at quote	Same, except under REROUTE	Consistent
Bank fee → shortfall → top-up	carry_forward is the FRS behaviour; waived / absorbed added	Per NPL instruction
Registry-only destinations	SETTLEMENT_REGISTRATION.approval_status — FRS project has all approved	Generalisation
AQN-LT-Sub offset-settled collections	OWN_WALLET at Aquanow; pass-through is a two-leg deal group or a reroute via the own wallet; also NPL-GR's own-desk collection endpoint	Resolved (v4.0, D23)

Draft v5.3 — internal baseline. Dependent documents (ledger design, calculation spec, state machines, roles matrix, configuration schema, reading guide, fee outlines) are still at their v5.2 editions and are to be regenerated against this version per §6.

Appendix B · Data dictionary (field annotations)

Every table also carries `created_at` and `created_by`. Types: int, string, bool, decimal, datetime, date. Rows changed in a revision are marked with the revision and decision that changed them.

B.1 Configuration

PROJECT

Field	Type	Key	Notes
id	int	PK	
name	string		
client_label	string		
quote_freshness_min	int		
quote_validity_min	int		
grace_min	int		
grace_drift_pct	decimal		
late_collection_policy	string		requote / honor_with_approval
bank_fee_treatment_default	string		waived / carry_forward / absorbed
receiver_fee_timing	string		at_conversion (default, Q6) / post_confirmation
network_fee_policy	string		absorbed / charged_back (project default; per-transaction override via FEE_DECISION)
disburse_policy	string		to_zero / hold_allowed (Q5)
shortfall_tolerance	decimal		
exposure_ceiling	decimal		cap on balance resting at a partner
status	string		

PARTNER

Field	Type	Key	Notes
id	int	PK	
name	string		Ali, Jeton, PT
disclosure_style	string		markup_included / market_plus
rate_expiry_time	string		
holiday_calendar	string		
status	string		
bank_cutoff_time	time		the partner's daily bank cutoff for pairs that need a bank lock (Ali 14:30) (v5.3, D29 amended 9 Oct — moved from PARTNER_CONFIG)
cutoff_timezone	string		IANA zone, e.g. Asia/Hong_Kong (v5.3, D29)

RATE_SOURCE

Field	Type	Key	Notes
id	int	PK	
code	string		KRAKEN / CMC / OANDA / XE
staleness_threshold_s	int		

INTRODUCER

Field	Type	Key	Notes
id	int	PK	
name	string		
status	string		

SENDER

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
name	string		Ayush, Naveen
risk_tier	string		
status	string		

RECEIVER_GROUP

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
name	string		
is_default	bool		exactly one per project
status	string		

RECEIVER

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
group_id	int	FK	receiver group; null unless kind = counterparty (v5.0, D13)
kind	string		counterparty / sender_return / partner_transit
sender_id	int	FK	when kind = sender_return
partner_config_id	int	FK	when kind = partner_transit
name	string		the group: Sud, Raj
status	string		

OWN_WALLET

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
provider	string		e.g. Aquanow (Q10)
account_label	string		e.g. LT Sub
network	string		
address	string		
signer_arrangement	string		hardware / multisig
custody_max_hours	int		

THRESHOLD

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
kind	string		max_amount / settlement_limit / daily_slots / rate_variation_rel / custom
currency	string		
value	decimal		
condition_expr	string		custom conditions
escalation	string		finance / management

CURRENCY_PAIR

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
from_currency	string		
to_currency	string		
from_cash_location_id	int	FK	nullable; set iff the from side is collected as cash — the pair reads INR (Cash-Delhi) → USDT (v5.3, D30 amended 9 Oct)
to_cash_location_id	int	FK	nullable; set iff the to side is paid out as cash — USDT → JPY (Cash-Tokyo) (v5.3, D30 amended 9 Oct)
quote_direction	string		out_per_in / in_per_out
rate_precision	int		
amount_rounding	string		truncate_unit / nearest_unit / round_up_unit / nearest_n / round_up_n (v5.3, D25; was truncate_whole_unit only)
cash_rounding	string		truncate_n / nearest_n / round_up_n — how the collected cash amount is rounded to cash_rounding_unit (v5.3, D25; was nearest only)
cash_rounding_unit	int		100 / 500 / 1000 by currency — the unit the collected cash amount is rounded to
status	string		
rounding_unit	decimal		required for nearest_n / round_up_n (1,000 for THB, INR, JPY payouts) (v5.3, D25)

PARTNER_CONFIG

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
partner_id	int	FK	
applicable_for_reroute	bool		
status	string		
is_own_desk	bool		default false; NPL converts in its own wallet for this project (NPL-GR); enabling needs Management approval (v5.3, D23)

COLLECTION_SENDING_ENTITY

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	

Field	Type	Key	Notes
label	string		sender's own account or a client's account
account_ref	string		

SENDER_RECEIVER_ALLOW

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
receiver_group_id	int	FK	sendable_to (was receiver_id)

RECEIVING_ENTITY

Field	Type	Key	Notes
id	int	PK	
receiver_id	int	FK	
name	string		NewXP Entity, ReferScout Entity
status	string		

PAIR_RATE_SOURCE

Field	Type	Key	Notes
id	int	PK	
currency_pair_id	int	FK	
rate_source_id	int	FK	
priority	int		

PARTNER_PAIR

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
currency_pair_id	int	FK	
partner_pricing	string		market_plus_pct / disclosed_rate
partner_markup_pct	decimal		known only for market_plus
status	string		
needs_bank_lock	bool		conversion depends on the banking channel and the partner's cutoff (Evo pairs into EUR; USDT → USD / SGD at Ali) (v5.3, D29)

SETTLEMENT_SENDING_ENTITY

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
narrative_name	string		what the receiver sees
status	string		

SETTLEMENT_RAIL

Field	Type	Key	Notes
id	int	PK	

Field	Type	Key	Notes
partner_config_id	int	FK	
name	string		bank or EMI executing the transfer
bank_details_ref	string		
expected_bank_fee	decimal		
currency	string		

FEE_OVERRIDE

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
partner_pair_id	int	FK	
pct	decimal		
effective_from	datetime		
approved_by	int	FK	
sender_share_pct	decimal		optional; with receiver_share_pct replaces the structure's proportion (v5.3, D20)
receiver_share_pct	decimal		optional; both blank → structure's proportion; both set → sum = pct (v5.3, D20)

RECEIVING_ENTITY_ACCOUNT

Field	Type	Key	Notes
id	int	PK	
receiving_entity_id	int	FK	
currency	string		
rail	string		SWIFT / SEPA / TRC-20 / ERC-20 / ...
account_kind	string		bank / wallet (v5.0, D14)
bank_details_ref	string		external store; when account_kind = bank (was details_ref)
network	string		when account_kind = wallet
address	string		when account_kind = wallet; checksum-validated per network
status	string		
usage	string		standing / single_use — a local account the receiver gives for one payout (v5.3, D32)
reserved_for_deal_id	int	FK	nullable; the one deal a single_use account serves (v5.3, D32)

SETTLEMENT_REGISTRATION

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
receiving_entity_account_id	int	FK	was receiving_entity_id
approval_status	string		pending_partner / approved / suspended
bank_details_ref	string		as paid by this partner; only when the account's kind is bank (v5.2, D16)
network	string		as registered with this partner; only when the account's kind is wallet (v5.2, D16)

Field	Type	Key	Notes
address	string		as registered with this partner; only when the account's kind is wallet (v5.2, D16)
approved_at	datetime		
wallet_screening_id	int	FK	nullable; the receiver-side screening of a wallet account (v5.3, D31)

FEE_STRUCTURE

Field	Type	Key	Notes
id	int	PK	
partner_pair_id	int	FK	
rate_basis	string		market / partner
mode	string		fixed / variable
pct	decimal		
sender_share_pct	decimal		
receiver_share_pct	decimal		
floor_pct	decimal		
cap_pct	decimal		
min_margin_pct	decimal		
variance_threshold_rel	decimal		
quoting_rate_policy	string		worst_applicable / designated / best_applicable
effective_from	datetime		
displayed_fee_basis	string		actual / sender_share_on_partner_rate / fixed (v5.3, D22)
displayed_fee_pct	decimal		when displayed_fee_basis = fixed (BF via Ali shows 1 %) (v5.3, D22)

REFERRAL_RULE

Field	Type	Key	Notes
id	int	PK	
introducer_id	int	FK	
project_id	int	FK	
partner_pair_id	int	FK	optional
basis	string		share_of_earnings / fixed_pct / share_of_net_markup (v5.3, D26: base = fee parts – partner cost on the leg, on amount × market; rebates never in the base)
pct	decimal		
effective_from	datetime		
fee_structure_id	int	FK	optional scope: only deals priced on this structure (Raeen sub-account 50 %, Raeen direct downlines 75 %) (v5.3, D26)
party_kind	string		introducer / client_party (v5.3, D26)
settlement_mode	string		npl_pays / party_retains — the party holds the converted funds and pays NPL the remainder monthly (v5.3, D26)

PARTNER_REBATE_RULE

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
effective_from	datetime		

Field	Type	Key	Notes
basis	string		pct_of_partner_fee / pct_of_amount (Jeton: 0.4 % of amount on a 1 % or 1.25 % fee, 0.3 % for Novi's 0.9 %) (v5.3, D27)
pct	decimal		the rebate in the chosen basis (v5.3, D27)
fee_structure_id	int	FK	optional scope (v5.3, D27)
sender_id	int	FK	optional scope (Novi) (v5.3, D27)

PARTNER_ENTITY

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
name	string		the partner's legal vehicle: PT-tour, PT Global Inc
status	string		

COLLECTION_RECEIVING_ENDPOINT

Field	Type	Key	Notes
id	int	PK	
partner_entity_id	int	FK	the vehicle whose name is on the account or wallet
kind	string		bank / wallet
currency	string		
rail	string		nullable (v5.2, D18): a bank endpoint needs it to go active; optional for wallets, must match network when set
bank_details_ref	string		external store; when kind = bank
network	string		TRC-20 / ERC-20 / ...; when kind = wallet
address	string		when kind = wallet; checksum-validated per network
status	string		
own_wallet_id	int	FK	nullable; set when the endpoint is NPL's own wallet under an own-desk configuration (v5.3, D23)
risk_band	string		standard / elevated — which screening decisions may land here (v5.3, D31)
usage	string		standing / single_use (ad hoc local deposit accounts) (v5.3, D32)
reserved_for_deal_id	int	FK	nullable; the one deal a single_use endpoint serves (v5.3, D32)
expected_amount	decimal		nullable; the amount a single_use endpoint was arranged for (v5.3, D32)

FEE_TIER

Field	Type	Key	Notes
id	int	PK	
fee_structure_id	int	FK	
min_monthly_volume	decimal		threshold in volume_currency, per calendar month
volume_currency	string		
pct	decimal		
sender_share_pct	decimal		optional
receiver_share_pct	decimal		optional
effective_from	datetime		

WALLET_SCREENING

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
subject_kind	string		sender_wallet / receiver_wallet
sender_id	int	FK	when subject_kind = sender_wallet
receiving_entity_account_id	int	FK	when subject_kind = receiver_wallet
network	string		
address	string		
provider_ref	string		screening tool reference
result	string		standard / elevated / unacceptable
decision	string		accept / accept_alternate / reject
screened_by	int	FK	Operations
screened_at	datetime		
evidence_ref	string		

CASH_LOCATION

Field	Type	Key	Notes
id	int	PK	
currency	string		
city	string		Delhi, Mumbai, Bangkok, Singapore
country	string		
status	string		

B.2 Rates**MARKET_RATE**

Field	Type	Key	Notes
id	int	PK	
rate_source_id	int	FK	
currency_pair_id	int	FK	
rate	decimal		
captured_at	datetime		

RATE_COMPARISON

Field	Type	Key	Notes
id	int	PK	
partner_pair_id	int	FK	
rate_date	date		
avg_spread_2w	decimal		
avg_spread_4w	decimal		
status	string		open / closed
created_at	datetime		created lazily by the first deal of the day

PARTNER_RATE_VERSION

Field	Type	Key	Notes
id	int	PK	
rate_comparison_id	int	FK	
version_no	int		
rate	decimal		normalised to CURRENCY_PAIR.quote_direction; derived from quoted_rate (v5.3, D28)
market_rate_id	int	FK	
spread_pct	decimal		
status	string		active / usable / breached / approved / rejected
confirmed_by	int	FK	
approval_reason	string		
expires_at	datetime		
quoted_rate	decimal		as the partner stated it (Ali: 1.1738 EUR→USDT style) (v5.3, D28)
quoted_direction	string		out_per_in / in_per_out; rate = quoted_rate, or $1 \div \text{quoted_rate}$ at the pair's precision when the directions differ — derived, never typed (v5.3, D28)

B.3 Deal, collection, conversion, reroute, balance conversion**DEAL_GROUP**

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
sender_id	int	FK	
receiver_group_id	int	FK	entitled group
client_ref	string		D-YYMMDD-seq, what the sender sees
from_currency	string		
to_currency	string		
sender_rate	decimal		
fee_structure_id	int	FK	
amount_in	decimal		
target_out	decimal		
entry_mode	string		quote_first / collect_first / estimate_first
state	string		
quoted_at	datetime		
displayed_fee_pct	decimal		stamped at quote (v5.3, D22)
displayed_source_rate	decimal		= sender_rate $\div$ (1 – displayed_fee_pct); the 'market rate' the quote prints (v5.3, D22)
invoice_id	int	FK	nullable; the obligation this group pays (v5.3, D24)

DEAL

Field	Type	Key	Notes
id	int	PK	
deal_group_id	int	FK	every deal belongs to a group
leg_no	int		

Field	Type	Key	Notes
leg_type	string		forward / return
funding_source	string		collection / returned_disbursement / prior_leg
funding_ref_id	int		DISBURSEMENT_RETURN id or prior DEAL id
project_id	int	FK	
sender_id	int	FK	
receiver_group_id	int	FK	attribution (D1)
receiver_id	int	FK	optional until settlement
partner_pair_id	int	FK	
partner_entity_id	int	FK	
amount_mode	string		target_out / fixed_in
collection_method	string		crypto / bank / cash — cash iff the pair's from side names a CASH_LOCATION (D19, D30)
cash_instructions	string		free text for the hand-over; the city is on the pair, the token on the collection (D30)
fee_structure_id	int	FK	
market_rate_id	int	FK	
partner_rate_version_id	int	FK	
sender_rate	decimal		
amount_in	decimal		
gross_out	decimal		
client_net	decimal		
fee_sender_part	decimal		
fee_receiver_part	decimal		
priced_at	datetime		quote or conversion
sent_at	datetime		null on conversion-priced legs
valid_until	datetime		
grace_until	datetime		
late_decision	string		none / honored / requoted
state	string		
quote_hash	string		
fee_override_id	int	FK	nullable; the override that priced this leg (v5.3, D20)
fee_tier_id	int	FK	nullable; base tier at quote (v5.3, D21)
tier_volume_basis	decimal		project month-to-date volume that justified the tier, stamped at quote (v5.3, D21)
rate_locked_at	datetime		when the partner confirmed the rate; before the cutoff → converts the same day, after → next business day (v5.3, D29)
conversion_due_date	date		derived from rate_locked_at and the partner's cutoff; Converting may not start earlier (v5.3, D29)
conversion_deferred_reason	string		set when the due date is pushed because the next day's rate breaches the minimum margin (ask for a better rate, or wait a day) (v5.3, D29)

COLLECTION

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	
collection_receiving_endpoint_id	int	FK	the partner endpoint paid into; null for cash (v5.1, D15; was endpoint_type + endpoint_id)
collection_sending_entity_id	int	FK	
amount	decimal		
evidence_ref	string		hash / slip / receipt
state	string		pending / verified / reversed (v5.3, D32); cash: token_issued → handed_over → counted → verified (D30)
verified_by	int	FK	
verified_at	datetime		
sending_address	string		crypto: the sender's declared wallet (v5.3, D31)
wallet_screening_id	int	FK	nullable; the screening that admitted the sending wallet (v5.3, D31)
screening_skipped	bool		projects that do not screen every sender up front (Evo) (v5.3, D31)
token_ref	string		cash: serial of the token (local-currency bill) that identifies the agent (v5.3, D30)
agent_contact_ref	string		cash: the receiving agent's contact, external store (v5.3, D30)
counted_at	datetime		cash: the partner counted and confirmed the funds (v5.3, D30)

CONVERSION

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	null when owned by a balance conversion
balance_conversion_id	int	FK	null when owned by a deal leg
partner_rate_version_id	int	FK	
calc_mode	string		npl_calculates / partner_calculates
amount_in	decimal		
expected_out	decimal		
actual_out	decimal		booked figure, rounded as the pair says (D25)
rounding_residual	decimal		signed (v5.3, D25): positive = fraction dropped, stays client money; negative = amount added by rounding up, NPL's cost posted to EXP_ROUNDING
fee_receiver_part	decimal		taken at conversion (Q6)
variance	decimal		
residual_in	decimal		margin pool
basis_variance	decimal		
state	string		
approved_by	int	FK	

REROUTE

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	
trigger	string		partner_unavailable / limit_reached / holiday / compliance_hold / other
phase	string		pre_collection / post_collection
outcome	string		via_own_wallet / refund_sender (Q3, post-collection)
own_wallet_id	int	FK	
alternate_partner_config_id	int	FK	
forwarding_policy	string		forward_full / forward_and_recover_dues
recovered_amount	decimal		≤ outstanding receivable at the alternate partner
partner_agreement_ref	string		the partner agreed the net amount
reason	string		
requested_by	int	FK	
approved_by	int	FK	

SENDER_CREDIT

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
deal_id	int	FK	
amount	decimal		
state	string		open / applied

EARNINGS_RECEIVABLE

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
deal_id	int	FK	
amount	decimal		
state	string		outstanding / recovered
recovered_by_deal_id	int	FK	
recovered_at	datetime		

BALANCE_CONVERSION

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
partner_config_id	int	FK	
receiver_group_id	int	FK	group served; chosen split
from_currency	string		
to_currency	string		
partner_pair_id	int	FK	

Field	Type	Key	Notes
partner_rate_version_id	int	FK	
market_rate_id	int	FK	
amount_in	decimal		
expected_out	decimal		
actual_out	decimal		
variance	decimal		
rounding_residual	decimal		
state	string		
requested_by	int	FK	Finance
approved_by	int	FK	Management
converted_at	datetime		
rate_locked_at	datetime		(v5.3, D29)
conversion_due_date	date		banking-ready balances (USD, SGD) wait for the next business morning (v5.3, D29)

ENTITLEMENT_REATTRIBUTION

Field	Type	Key	Notes
id	int	PK	
balance_conversion_id	int	FK	
receiver_group_id	int	FK	
amount_from	decimal		
amount_to	decimal		

DEAL_FEE_TIER

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	
fee_tier_id	int	FK	
amount_in_portion	decimal		Σ over a leg = amount_in
pct	decimal		the tier's pct at quote

INVOICE

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
receiver_id	int	FK	the vendor owed
currency	string		
amount	decimal		
reference	string		
issued_at	datetime		
status	string		open / settled / void

B.4 Disbursement, return, confirmation, fees, referrals, rebates

DISBURSEMENT

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
partner_config_id	int	FK	
settlement_sending_entity_id	int	FK	
settlement_rail_id	int	FK	
currency	string		
total	decimal		rounded as the pair says (D25)
bank_fee_treatment	string		
state	string		
prepared_by	int	FK	
approved_by	int	FK	
escalated_to	int	FK	
released_at	datetime		

DISBURSEMENT_LINE

Field	Type	Key	Notes
id	int	PK	
disbursement_id	int	FK	
deal_id	int	FK	nullable; required when the paid receiver is group-less, optional on counterparty lines (v5.2, D17)
settlement_registration_id	int	FK	
paid_receiver_id	int	FK	
amount	decimal		
receiver_fee_applied	decimal		exceptional; fee is normally taken at conversion
reference	string		entity invoice format
outcome	string		sent / confirmed / returned
invoice_id	int	FK	nullable; the obligation this line discharges (v5.3, D24)
obligation_discharged	decimal		in the invoice currency (v5.3, D24)
discharge_rate	decimal		payout currency per invoice currency at payment (v5.3, D24)
discharge_market_rate_id	int	FK	nullable; the market snapshot used (v5.3, D24)
token_ref	string		cash settlement: serial of the token handed to the receiver's agent (v5.3, D30)

DISBURSEMENT_RETURN

Field	Type	Key	Notes
id	int	PK	
disbursement_line_id	int	FK	
returned_amount	decimal		
bank_fee	decimal		
reason	string		
returned_at	datetime		

CONFIRMATION

Field	Type	Key	Notes
id	int	PK	
disbursement_line_id	int	FK	
amount_received	decimal		
slip_ref	string		
result	string		full / short
confirmed_by	int	FK	
confirmed_at	datetime		

SHORTFALL

Field	Type	Key	Notes
id	int	PK	
receiver_id	int	FK	
confirmation_id	int	FK	
amount	decimal		
state	string		open / topped_up
topped_up_disbursement_id	int	FK	

BANK_FEE_EVENT

Field	Type	Key	Notes
id	int	PK	
disbursement_id	int	FK	
expected_amount	decimal		
actual_amount	decimal		
treatment	string		waived / carry_forward / absorbed
state	string		open / applied
applied_disbursement_id	int	FK	

FEE_DECISION

Field	Type	Key	Notes
id	int	PK	
subject_type	string		Deal / Disbursement / BalanceConversion
subject_id	int		
fee_kind	string		markup / network / bank
decision	string		apply / waive / override
pct	decimal		
approved_by	int	FK	
reason	string		

REFERRAL_ACCRUAL

Field	Type	Key	Notes
id	int	PK	
referral_rule_id	int	FK	

Field	Type	Key	Notes
deal_id	int	FK	
amount	decimal		
state	string		accrued / reconciled / settled (v5.3, D26; was accrued / paid)
paid_at	datetime		
direction	string		payable (NPL pays the party) / receivable (the party holds the funds and owes NPL its remainder) (v5.3, D26)
period	string		YYYY-MM; receivables reconcile monthly like rebates (v5.3, D26)
received_amount	decimal		on receivables (v5.3, D26)

PARTNER_REBATE_ACCRUAL

Field	Type	Key	Notes
id	int	PK	
partner_rebate_rule_id	int	FK	
deal_id	int	FK	
expected_amount	decimal		
received_amount	decimal		
period	string		YYYY-MM
state	string		expected / reconciled / received
statement_ref	string		the partner statement line it reconciled against (v5.3, D27)

LOSS_EVENT

Field	Type	Key	Notes
id	int	PK	
side	string		collection / settlement
collection_id	int	FK	when side = collection
disbursement_line_id	int	FK	when side = settlement
amount	decimal		≤ the verified or released amount
reason	string		account_frozen / cash_short / wrong_chain / other
occurred_at	datetime		
sender_share	decimal		
npl_share	decimal		
partner_share	decimal		
client_share	decimal		shares sum to amount
state	string		open / approved / posted
approved_by	int	FK	Management

B.5 Ledger & controls**LEDGER_TRANSACTION**

Field	Type	Key	Notes
id	int	PK	
source_type	string		Collection / Conversion / BalanceConversion / Disbursement / DisbursementReturn / Confirmation / BankFeeEvent / Reroute / Referral / Rebate / LossEvent / Adjustment

Field	Type	Key	Notes
source_id	int		
description	string		
posted_at	datetime		
posted_by	int	FK	
reversal_of_id	int	FK	

LEDGER_POSTING

Field	Type	Key	Notes
id	int	PK	
ledger_transaction_id	int	FK	
ledger_account_id	int	FK	
side	string		debit / credit
amount	decimal		
currency	string		
ownership_tag	string		client / company
cost_component	string		principal / earnings / rebate / partner_cost / bank_fee / network_fee / variance / referral / share / rounding / fx_timing / cutoff_timing / loss (v5.3: + rebate, share, fx_timing, cutoff_timing, loss)

LEDGER_ACCOUNT

Field	Type	Key	Notes
id	int	PK	
code	string		OWNER.HOLDER.CCY.PURPOSE
ownership	string		client / company
holder_type	string		PartnerConfig / OwnWallet / Receiver / Sender / Introducer / Project / None
holder_id	int		
currency	string		
purpose	string		COLLECTED / HELD / DUE / INTRANSIT / PAYABLE / SHORTFALL / CREDIT / POOL / WALLET / EARN_GROSS / EARN_REBATE / EXP_PARTNER / EXP_BANKFEE / EXP_NETWORK / EXP_ROUNDING / EXP_LOSS / VARIANCE / REFERRAL_PAYABLE / SHARE_RECEIVABLE / REBATE_RECEIVABLE
kind	string		asset / liability / income / expense

USER

Field	Type	Key	Notes
id	int	PK	
email	string		Google Workspace
role	string		ops / finance / management
status	string		

APPROVAL

Field	Type	Key	Notes
id	int	PK	
subject_type	string		
subject_id	int		

Field	Type	Key	Notes
action	string		
requested_by	int	FK	
approved_by	int	FK	never equal to requested_by
escalated_to	int	FK	
state	string		
reason	string		

AUDIT_LOG

Field	Type	Key	Notes
id	int	PK	
record_type	string		
record_id	int		
field	string		
old_value	string		
new_value	string		
actor_id	int	FK	
at	datetime		

RECORD_LOCK

Field	Type	Key	Notes
id	int	PK	
record_type	string		
record_id	int		
locked_by	int	FK	
locked_at	datetime		